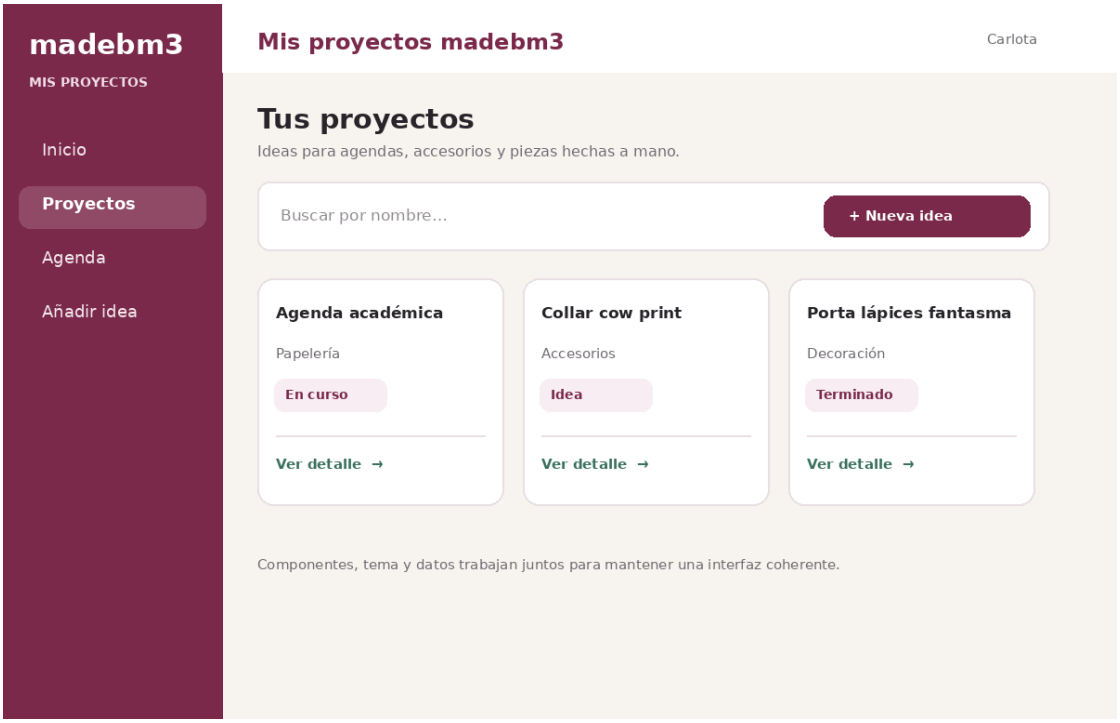
GUÍA PRÁCTICA PARA APRENDER REACT

REACT DESDE CERO HASTA PUBLICAR

Manual completo de desarrollo con el proyecto madebm3

Componentes · rutas · formularios · interfaz · APIs · pruebas · publicación



Un único recorrido, organizado en cinco secciones

Cómo usar este manual

Este manual reúne en un solo recorrido los contenidos que estaban distribuidos en cinco documentos. Cada sección amplía la misma aplicación madebm3: un panel para organizar agendas, accesorios y piezas hechas a mano.

Sigue las secciones en orden y trabaja sobre el mismo proyecto Vite. Los ejemplos de código muestran una forma de construir cada pieza; adáptalos a los nombres y componentes que ya tengas en tu app. Haz una copia del proyecto antes de incorporar cambios grandes.

Ruta de aprendizaje

SECCIÓN 1 · Fundamentos de React
Aprende cómo React convierte datos en interfaces y construye el primer panel madebm3.

SECCIÓN 2 · Navegación y formularios
Organiza el panel en rutas y mejora los flujos para crear, editar y buscar proyectos.

SECCIÓN 3 · Interfaz y librerías
Da coherencia visual a madebm3 con Material UI, un tema y componentes reutilizables.

SECCIÓN 4 · Datos y servicios
Conecta las pantallas a una API local de práctica y aprende a guardar datos con HTTP.

SECCIÓN 5 · Calidad y publicación
Prueba los comportamientos importantes, revisa accesibilidad y prepara un build de producción.

La API local del apartado de datos sirve para practicar en tu ordenador. No es un backend público ni se debe usar para guardar datos reales cuando publiques la interfaz.

Contenido

1. Fundamentos de React
Aprende cómo React convierte datos en interfaces y construye el primer panel madebm3.

2. Navegación y formularios
Organiza el panel en rutas y mejora los flujos para crear, editar y buscar proyectos.

3. Interfaz y librerías
Da coherencia visual a madebm3 con Material UI, un tema y componentes reutilizables.

4. Datos y servicios
Conecta las pantallas a una API local de práctica y aprende a guardar datos con HTTP.

5. Calidad y publicación
Prueba los comportamientos importantes, revisa accesibilidad y prepara un build de producción.

SECCIÓN 1

Fundamentos de React

Aprende cómo React convierte datos en interfaces y construye el primer panel madebm3.

Al terminar esta sección

Al terminar podrás dividir una interfaz en componentes, mover datos con props, manejar estado y guardar una primera versión local.

Apartados

1.1 1. Qué es React
1.2 JavaScript mínimo para seguir el libro
1.3 Prepara el espacio de trabajo
1.4 JSX: escribir la interfaz
1.5 Componentes: divide para entender
1.6 Props: pasar datos a una pieza
1.7 Listas, map y la prop key
1.8 Mostrar estados y pantallas vacías
1.9 Estado y eventos con useState
1.10 Formularios que controlas desde React
1.11 Actualizar, borrar y filtrar proyectos
1.12 Estilos y diseño que se adapta
1.13 Efectos y guardado local
1.14 El flujo de datos en React
1.15 Leer errores y trabajar con orden
1.16 Proyecto final: Mis proyectos madebm3
1.17 Estructura orientativa del proyecto
1.18 Repaso y continuación

APARTADO 1.1

Qué es React

React es una biblioteca de JavaScript para construir interfaces. En vez de escribir una pantalla como un bloque monolítico, la divides en componentes: piezas que muestran una parte de la aplicación y que se pueden reutilizar.

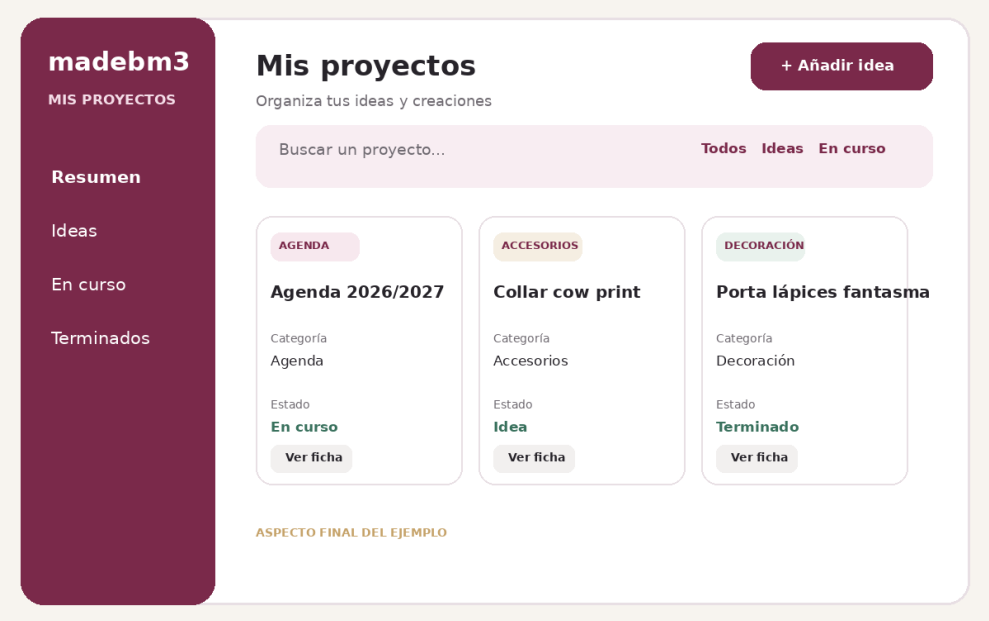
La idea central

Tu interfaz depende de datos. Si los datos cambian, React vuelve a representar la parte que corresponde. Por ejemplo, la misma tarjeta puede mostrar una agenda “En curso” y un collar “Idea” porque recibe información diferente.

```
function Greeting() {
  return <h1>Hola, Carlota</h1>;
}
```

Qué produce este ejemplo

En el navegador verás una página con un título grande que dice “Hola, Carlota”. El código describe esa salida; React se encarga de actualizarla y colocarla en la página.



Una interfaz de proyectos puede construirse combinando muchas piezas pequeñas.

React no es toda la aplicación

React resuelve la construcción de la interfaz. Para el proyecto inicial, Vite prepara los archivos y ejecuta un servidor local. Más adelante, una app puede sumar navegación, conexión con servicios, pruebas u otras herramientas. No necesitas incorporar todo eso para aprender los fundamentos.

APARTADO 1.2

JavaScript mínimo para seguir el libro

React está escrito con JavaScript. Estos patrones aparecen continuamente en los ejemplos.

Variables y valores

Usa const para un valor que no vas a reasignar y let cuando sí necesites reasignarlo. Una cadena de texto se escribe entre comillas; un número, sin ellas.

```
const title = "Collar cow print";
const price = 18;
let isFinished = false;
```

Objetos y arrays

Un objeto agrupa propiedades relacionadas. Un array contiene una lista ordenada de valores; normalmente cada proyecto será un objeto dentro de un array.

```
const project = {
  id: 1,
  title: "Agenda académica",
  category: "Papelería",
  status: "En curso",
};

const projects = [project];
```

Funciones y listas

Una función agrupa instrucciones. En React se usa mucho la forma con flecha. map transforma una lista y filter conserva solo los elementos que cumplen una condición; ambos dejan intacto el array original.

```
const getLabel = (project) => project.title;
const titles = projects.map((project) => project.title);
const ideas = projects.filter((project) => project.status === "Idea");
```

Lectura del código project es el nombre temporal que recibe cada elemento de la lista mientras map o filter lo recorren. Puedes elegir otro nombre, pero conviene que describa qué contiene.

Práctica Crea tres objetos de proyecto con títulos y estados distintos. Usa map para obtener una lista de sus títulos.

APARTADO 1.3

Prepara el espacio de trabajo

Primero crea un proyecto local con React y Vite. Este flujo lo usarás para probar cada apartado.

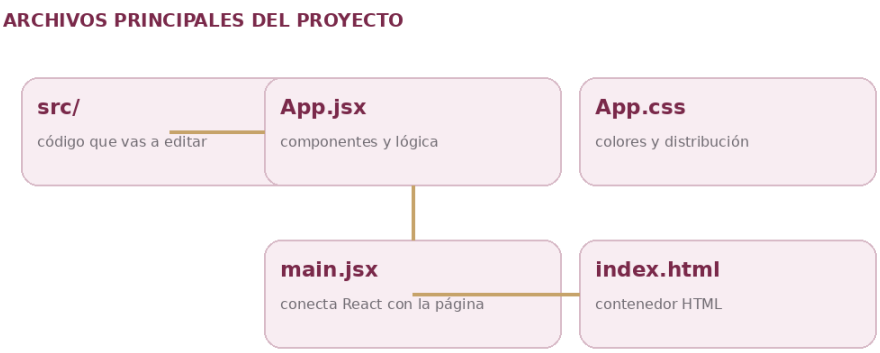
Instalación paso a paso

1. Instala Node.js desde la página oficial y elige la versión LTS.
2. Abre VS Code y selecciona Terminal → New Terminal.
3. Comprueba las herramientas: node -v y npm -v.
4. Crea el proyecto: npm create vite@latest madebm3-react -- --template react.
5. Entra en la carpeta: cd madebm3-react.
6. Instala dependencias: npm install.
7. Inicia el servidor: npm run dev.
8. Abre en el navegador la dirección local que aparezca en el terminal.

```
node -v
npm -v
npm create vite@latest madebm3-react -- --template react
cd madebm3-react
npm install
npm run dev
```

Qué archivo abrir

Para la mayoría de los ejercicios de este manual, editarás src/App.jsx y src/App.css. El terminal debe seguir abierto mientras uses la aplicación local. Cuando termines, puedes detener el servidor con Ctrl+C.



Mapa sencillo de los archivos que tocarás con más frecuencia.

Si algo no coincide

- Si node -v no responde, instala Node.js y vuelve a abrir el terminal.
- Si npm run dev falla, confirma que estás dentro de la carpeta madebm3-react.
- Si la página no carga, copia la dirección local exacta que muestra Vite.

Señal de que arrancó

```
VITE vX.X.X ready
Local: http://localhost:5173/
Abre la dirección que muestra tu terminal; el puerto puede variar.
```

Primera comprobación ¿Qué dirección local aparece en tu terminal? ¿Qué ves al abrirla?

APARTADO 1.4

JSX: escribir la interfaz

JSX se parece a HTML, pero forma parte del código JavaScript y tiene reglas propias.

Tu primer componente

```
function App() {
  const name = "Carlota";

  return (
    <main>
      <h1>Hola, {name}</h1>
      <p>Este es mi panel de proyectos.</p>
    </main>
  );
}

export default App;
```

Cómo leer las llaves

El texto dentro de etiquetas aparece directamente. Las llaves insertan una expresión JavaScript. En el ejemplo, {name} se sustituye por el valor de la variable.

Reglas prácticas

- Devuelve un único elemento principal; puedes agruparlo con <main>, <section> o un fragmento <>…</>.
- Cierra las etiquetas: <img /> y <input /> llevan cierre.
- Usa className en JSX para asignar clases de CSS.
- Para los eventos escribe onClick, onChange y onSubmit con mayúscula interna.
- Los nombres de componente empiezan con mayúscula: App, ProjectCard.

Condiciones dentro del JSX

```
{isFinished ? <span>Terminado</span> : <span>En curso</span>}
```

El operador ternario expresa “si se cumple esta condición, muestra A; si no, muestra B”. Para mostrar u ocultar sin alternativa puedes usar &&: {showDetails && Más información }.

Práctica Modifica App para que muestre un título, un párrafo y una lista de tres tipos de proyecto que quieres organizar.

APARTADO 1.5

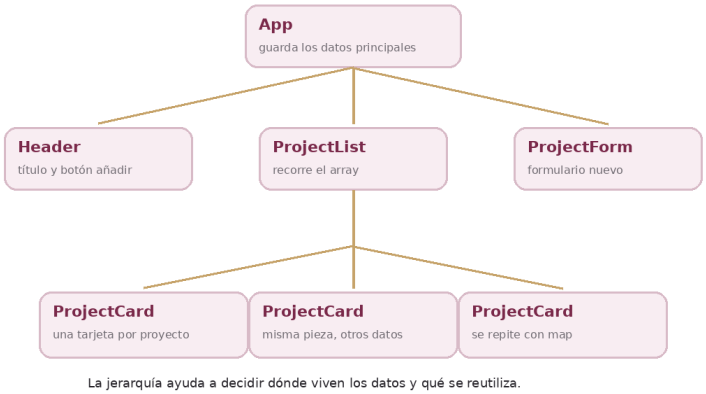
Componentes: divide para entender

Cuando una parte de la interfaz tiene una función clara, puede convertirse en un componente. Un componente no necesita ser grande; una tarjeta o un encabezado ya pueden serlo.

```
function App() {
  return <main><Header /><ProjectList /></main>;
}

function Header() {
  return <h1>Mis proyectos madebm3</h1>;
}

function ProjectList() {
  return <p>Aquí aparecerán tus proyectos.</p>;
}
```



La pantalla se compone de piezas anidadas. ProjectCard puede repetirse para cada elemento.

Cuándo separar una pieza

- Tiene un nombre que explica su propósito.
- Se repite en más de un lugar.
- Su JSX es tan largo que cuesta leer el componente principal.
- Tiene datos o una interacción propia que quieres mantener localizada.

Empieza pequeño Separa una pieza cuando se repita o cueste mantenerla dentro de App.

Práctica Crea Welcome y ProjectCard como funciones independientes. Úsalas desde App y confirma que ambas se muestran.

APARTADO 1.6

Props: pasar datos a una pieza

Las props son la información que un componente recibe desde otro componente. Son de solo lectura: la tarjeta las muestra, y el componente padre decide sus valores.

```
function ProjectCard({ title, category, status }) {
  return (
    <article className="card">
      <h2>{title}</h2>
      <p>Categoría: {category}</p>
      <p>Estado: {status}</p>
    </article>
  );
}

function App() {
  return (
    <ProjectCard
      title="Agenda 2026/2027"
      category="Papelería"
      status="En curso"
    />
  );
}
```

El recorrido de los datos

App pasa title, category y status. ProjectCard recibe esos mismos nombres y los coloca entre llaves en el JSX. Si el nombre que envías no coincide con el que lees, el dato aparecerá vacío o como undefined.

Personaliza la tarjeta Prueba props opcionales como dueDate o price. Si una tarjeta no siempre tiene fecha, puedes mostrarla solo cuando exista: {dueDate && Fecha: {dueDate} }.

Práctica Añade una tarjeta para un collar y otra para una pieza de yeso. Mantén el mismo componente y cambia las props.

APARTADO 1.7

Listas, map y la prop key

Los proyectos se guardan como datos en un array y se convierten en componentes con map. Así no escribes una tarjeta a mano por cada proyecto.

```
const projects = [
  { id: 1, title: "Agenda académica", category: "Papelería", status: "En curso" },
  { id: 2, title: "Collar cow print", category: "Accesorios", status: "Idea" },
  { id: 3, title: "Porta lápices fantasma", category: "Decoración", status: "Terminado" },
];

function ProjectList() {
  return (
    <section className="project-grid">
      {projects.map((project) => (
        <ProjectCard key={project.id} {...project} />
      ))}
    </section>
  );
}
```

Por qué importa key

key es una etiqueta que permite a React reconocer cada elemento aunque la lista cambie. Usa un id estable y único. No uses el índice de posición como key si vas a insertar, eliminar u ordenar proyectos.

El spread de propiedades

{...project} pasa las propiedades del objeto al componente. En los primeros ejercicios, puedes escribir title={project.title}, category={project.category} y status={project.status} de forma explícita; resulta más largo pero hace el recorrido muy visible.

Práctica Añade una cuarta tarjeta con un id que no esté repetido. ¿Qué advertencia aparece en consola si repites un id como key?

APARTADO 1.8

Mostrar estados y pantallas vacías

Una buena interfaz también explica qué ocurre cuando todavía no hay datos, cuando no hay resultados o cuando un proyecto cambia de estado.

```
function ProjectCard({ title, status }) {
  return (
    <article className="card">
      <h2>{title}</h2>
      <span className={status === "Terminado" ? "badge done" : "badge"}>
        {status}
      </span>
    </article>
  );
}

function ProjectList({ projects }) {
  if (projects.length === 0) {
    return <p>Aún no tienes proyectos. Añade tu primera idea.</p>;
  }
  return projects.map((project) => (
    <ProjectCard key={project.id} {...project} />
  ));
}
```

Tres situaciones a pensar

- Hay proyectos: muestra las tarjetas.
- La búsqueda no encuentra coincidencias: explica que no hay resultados.
- La lista está vacía: invita a crear la primera idea.

Práctica Añade un texto alternativo para el caso en que un filtro no devuelva ninguna tarjeta.

APARTADO 1.9

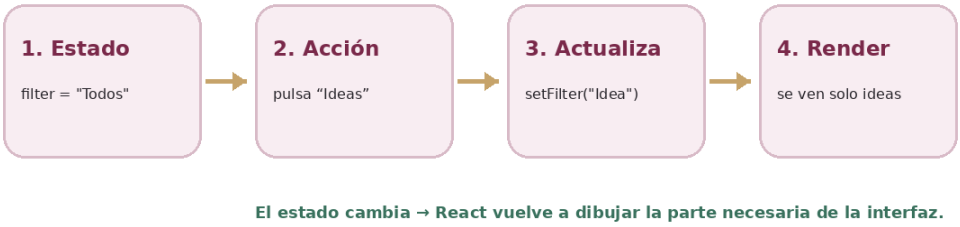
Estado y eventos con useState

El estado es un dato que puede cambiar durante el uso de la aplicación. useState le da a React un valor y una función para actualizarlo.

```
import { useState } from "react";

function Counter() {
 const [count, setCount] = useState(0);

 return (
  <section>
   <p>Ideas guardadas: {count}</p>
   <button onClick={() => setCount(count + 1)}>
    Añadir una
   </button>
  </section>
 );
}
```



El clic ejecuta una función, cambia el estado y provoca una nueva representación.

Qué significa cada línea

- useState(0) inicia el dato en cero.
- count es el valor actual en este render.
- setCount es la forma de pedir un valor nuevo.
- onClick recibe una función; no escribas setCount(count + 1) directamente como valor de onClick.

Cuando el cambio depende del valor anterior Puedes usar setCount(current => current + 1). Esta forma evita depender de una copia del valor que quizá haya quedado desactualizada.

Práctica Añade un botón “Quitar una” y evita que el contador baje de cero.

APARTADO 1.10

Formularios que controlas desde React

En un formulario controlado, el valor visible del campo está conectado al estado. Cada cambio del usuario actualiza el estado; al enviar, la aplicación usa ese valor.

```
const [title, setTitle] = useState("");

function handleSubmit(event) {
  event.preventDefault();
  if (!title.trim()) return;
  console.log("Nuevo proyecto:", title.trim());
  setTitle("");
}

<form onSubmit={handleSubmit}>
  <label htmlFor="project-title">Nombre del proyecto</label>
  <input
    id="project-title"
    value={title}
    onChange={(event) => setTitle(event.target.value)}
    placeholder="Ej. Bandeja decorativa"
  />
  <button type="submit">Guardar idea</button>
</form>
```

Accesibilidad básica

Asocia cada campo con un label. El texto de ayuda o error debe ser visible y claro. El usuario debe poder recorrer los controles con el teclado y entender qué hace cada botón.

Qué hace preventDefault

Los formularios HTML suelen recargar la página al enviarse. event.preventDefault() cancela ese comportamiento para que React gestione el envío.

Práctica Añade una categoría con un select. Guárdala en su propio useState, inclúyela en el objeto del nuevo proyecto y limpia ambos campos tras guardar.

APARTADO 1.11

Actualizar, borrar y filtrar proyectos

Para cambiar una lista guardada en estado, crea un nuevo array con map o filter. No mutas el array original con push, splice o asignaciones directas.

```
// Añadir
setProjects(current => [...current, newProject]);

// Eliminar el proyecto con el id recibido
setProjects(current => current.filter(project => project.id !== id));

// Marcar como terminado
setProjects(current => current.map(project =>
  project.id === id
    ? { ...project, status: "Terminado" }
    : project
));
```

Filtro derivado

```
const visibleProjects = projects.filter((project) => {
  const matchesStatus = filter === "Todos" || project.status === filter;
  const matchesSearch = project.title.toLowerCase().includes(query.toLowerCase());
  return matchesStatus && matchesSearch;
});
```

visibleProjects se calcula desde projects, filter y query. No hace falta guardarlo como estado separado porque puede derivarse de esos valores.



La interfaz cambia al añadir un objeto al array de proyectos.

Práctica Implementa un botón para eliminar una tarjeta. Después crea un filtro por estado y una búsqueda por título.

APARTADO 1.12

Estilos y diseño que se adapta

React crea la estructura de la interfaz. CSS define el aspecto, el espacio, el color y la distribución.

```
.app {
  max-width: 960px;
  margin: 0 auto;
  padding: 32px 20px;
  color: #27242A;
  font-family: Arial, sans-serif;
}

.project-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
}

.card {
  padding: 18px;
  border: 1px solid #eadde3;
  border-radius: 16px;
  background: white;
}
```

Aplicar una clase desde JSX

```
<main className="app">
  <section className="project-grid">
    {/* tarjetas */}
  </section>
</main>
```

La palabra dentro de className debe coincidir con el selector de CSS. Importa el archivo con import "./App.css" si no está ya importado.

Diseño adaptable

repeat(auto-fit, minmax(220px, 1fr)) permite que la cuadrícula ajuste el número de columnas al ancho disponible. Para una pantalla estrecha puedes cambiar el espacio y tamaño del encabezado:

```
@media (max-width: 600px) {
  .app { padding: 20px 14px; }
  .card { padding: 14px; }
}
```

Revisa el resultado reduciendo la ventana del navegador y comprobando que los botones y textos siguen siendo legibles.

Paleta para practicar Vino #7A294A, rosa claro #F8EDF2, crema #F7F4EF, verde suave #39715D y texto #27242A. Puedes cambiarla por otra; cuida que haya contraste suficiente.

Práctica Añade un estilo para los proyectos terminados y otro para los botones. Comprueba el diseño en una ventana estrecha.

APARTADO 1.13

Efectos y guardado local

useEffect permite sincronizar React con algo externo al render, como el almacenamiento del navegador. En esta práctica guardaremos proyectos con localStorage.

```
import { useEffect, useState } from "react";

const starterProjects = [
  { id: 1, title: "Agenda académica", status: "En curso" },
];

function App() {
  const [projects, setProjects] = useState(() => {
    const saved = localStorage.getItem("madebm3-projects");
    return saved ? JSON.parse(saved) : starterProjects;
  });

  useEffect(() => {
    localStorage.setItem("madebm3-projects", JSON.stringify(projects));
  }, [projects]);
}
```

Qué significa la dependencia

El array [projects] al final indica que el efecto se ejecuta cuando projects cambia. JSON.stringify convierte los datos en texto para guardarlos; JSON.parse los convierte de nuevo al leerlos.

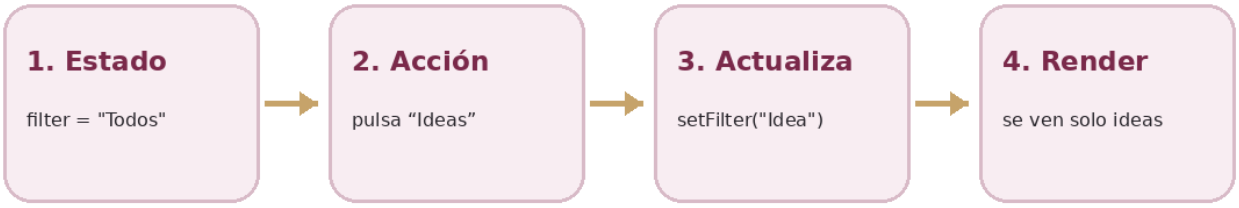
Límite de esta práctica localStorage solo guarda en ese navegador y dispositivo. No crea una cuenta, no sincroniza entre ordenadores y no es una base de datos segura para información privada.

Práctica Añade un proyecto, recarga la página y confirma que se conserva. Añade un botón “Restaurar ejemplos” para volver a starterProjects.

APARTADO 1.14

El flujo de datos en React

Una de las formas más útiles de entender React es seguir el recorrido desde los datos hasta la interfaz y de vuelta a una acción del usuario.



El estado cambia → React vuelve a dibujar la parte necesaria de la interfaz.

Estado → interacción → actualización → nueva representación.

Preguntas de depuración

- ¿Qué componente guarda el dato que cambia?
- ¿Qué componente necesita recibirlo?
- ¿Qué evento ocurre antes de que el dato cambie?
- ¿Se crea un nuevo array u objeto al actualizar el estado?
- ¿El JSX usa el valor actualizado para decidir qué mostrar?

Evita estado duplicado

Si puedes calcular un dato a partir de otro, normalmente no hace falta guardarlo dos veces. Por ejemplo, visibleProjects puede calcularse desde projects y filter. Mantener una sola fuente ayuda a que los valores no se contradigan.

Explica con tus palabras Describe qué ocurre desde que pulsas “Ideas” hasta que solo ves las tarjetas con ese estado.

APARTADO 1.15

Leer errores y trabajar con orden

Los mensajes de error son pistas. Mira el terminal de Vite y la consola del navegador; revisa primero el primer error, porque los siguientes pueden ser consecuencia del primero.

Errores frecuentes

Pantalla en blanco

Abre la consola del navegador y busca el primer error. Revisa la línea que indica.

Unexpected token

Comprueba llaves, paréntesis y etiquetas JSX sin cerrar.

X is not defined

Busca errores de mayúsculas, importaciones o nombres que no coinciden.

No aparece un estilo

Confirma que App.css está importado y que className coincide con el selector.

El formulario recarga la página

Añade event.preventDefault() al comienzo de handleSubmit.

Advertencia por key

Pasa un id único y estable como key={project.id}.

Los cambios no se ven

Guarda el archivo y comprueba que el terminal ejecuta la carpeta correcta.

Método corto para encontrar un fallo Reproduce el problema, lee el primer error completo, revisa la línea y los nombres cercanos, cambia una cosa y prueba otra vez. No cambies cinco partes a la vez.

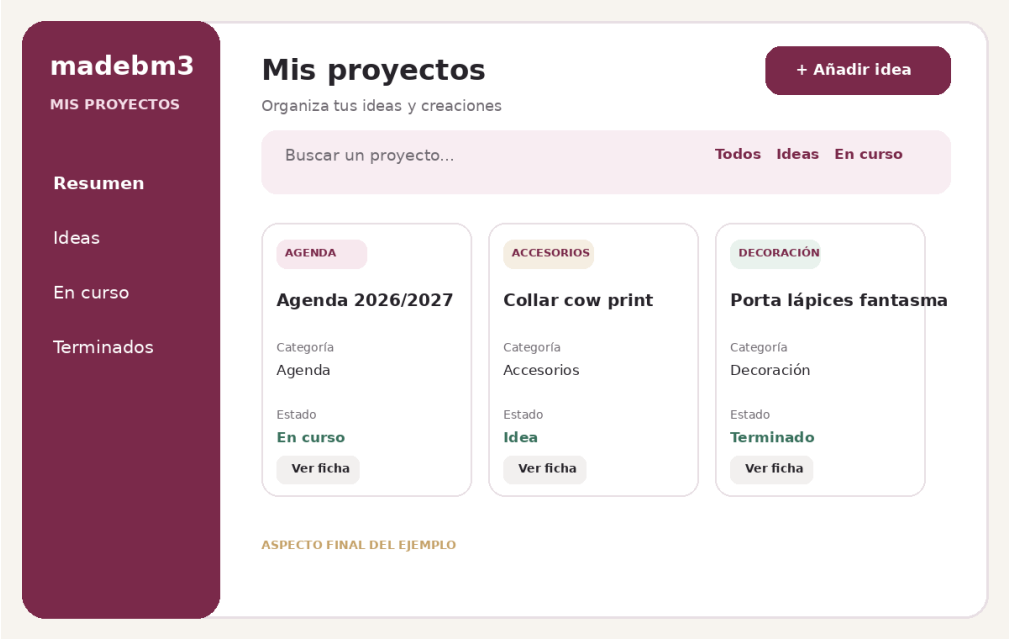
Bitácora de errores Escribe un error que hayas visto, qué significaba y cómo lo resolviste.

APARTADO 1.16

Proyecto final: Mis proyectos madebm3

El objetivo es combinar lo aprendido en un panel propio. Construye una función cada vez y prueba después de cada cambio.

Diseño de referencia



Maqueta visual de una posible versión final. Puedes cambiar sus colores y secciones.

Requisitos de la primera versión

- ☐ Encabezado y navegación sencilla.
- ☐ Tarjetas que muestran título, categoría y estado.
- ☐ Datos en un array con ids únicos.
- ☐ Filtro por estado y búsqueda por título.
- ☐ Formulario para añadir un proyecto.
- ☐ Acciones para terminar o eliminar un proyecto.
- ☐ Estilo adaptable a una pantalla estrecha.
- ☐ Persistencia de práctica con localStorage.

Orden recomendado

1. Construye el encabezado y una tarjeta estática.
2. Crea tres objetos y muéstralos con map.
3. Añade filtro y búsqueda; confirma qué datos cambian.
4. Añade formulario, validación de campo vacío y guardado.

5. Añade terminar y eliminar con actualizaciones inmutables.
6. Añade los estilos y prueba el diseño estrecho.
7. Recarga para comprobar la persistencia local.

Mi versión ¿Qué tres categorías usarás y qué estados tendrán tus proyectos?

APARTADO 1.17

Estructura orientativa del proyecto

Cuando el archivo principal crezca, puedes mover piezas a archivos propios. No es obligatorio para empezar; sirve para que el código resulte más fácil de encontrar.



Los archivos no son componentes: App.jsx puede contener varios componentes al principio.

```
src/
  App.jsx
  App.css
  components/
    Header.jsx
    ProjectCard.jsx
    ProjectForm.jsx
    ProjectList.jsx
```

Importar y exportar una pieza

```
// components/ProjectCard.jsx
export default function ProjectCard({ title }) {
  return <article className="card"><h2>{title}</h2></article>;
}

// App.jsx
import ProjectCard from "./components/ProjectCard";
```

Empieza con componentes en App.jsx para entender el recorrido. Cuando te resulte difícil localizar una pieza, muévela a su archivo y expórtala. Evita reorganizar carpetas antes de necesitarlo.

Práctica opcional Mueve ProjectCard a components/ProjectCard.jsx, impórtalo desde App.jsx y comprueba que la interfaz no cambia.

APARTADO 1.18

Repaso y continuación

Usa este repaso para comprobar qué conceptos ya puedes explicar sin mirar el libro.

Lista de control

- ☐ Entiendo qué es un componente y por qué se reutiliza.
- ☐ Puedo escribir JSX y poner expresiones dentro de llaves.
- ☐ Sé pasar información por props.
- ☐ Sé mostrar listas con map y keys estables.
- ☐ Sé manejar estado con useState.
- ☐ Sé crear un evento de clic y un formulario.
- ☐ Sé actualizar arrays sin mutarlos.
- ☐ Sé derivar una lista visible desde filtro y búsqueda.
- ☐ Sé añadir estilos con CSS y probar un diseño adaptable.
- ☐ Sé guardar datos de práctica con localStorage.
- ☐ Sé leer el primer error en consola o terminal.

Glosario esencial

Componente — Función que devuelve parte de la interfaz.

JSX — Sintaxis que permite escribir marcas parecidas a HTML dentro de JavaScript.

Props — Datos de solo lectura que un componente recibe.

Estado — Dato que puede cambiar y que React usa para actualizar la interfaz.

Render — Proceso de calcular qué interfaz corresponde a los datos actuales.

Hook — Función de React, como useState o useEffect, que habilita una capacidad en un componente.

Key — Identificador estable que React usa al mostrar elementos de una lista.

sección siguiente Cuando completes este proyecto, el siguiente bloque puede añadir navegación entre pantallas y formularios más completos. Entonces veremos qué librería elegir, qué problema resuelve y cómo instalarla y usarla.

SECCIÓN 2

Navegación y formularios

Organiza el panel en rutas y mejora los flujos para crear, editar y buscar proyectos.

Al terminar esta sección

Al terminar podrás declarar rutas, leer parámetros y validar formularios accesibles.

Apartados

2.1 De una pantalla a varias rutas
2.2 Añade React Router al proyecto
2.3 Declara tus primeras rutas
2.4 Navega con Link y NavLink
2.5 Comparte la estructura con Outlet
2.6 Abre el detalle con un parámetro
2.7 Guarda búsqueda y filtros en la URL
2.8 Comparte los proyectos entre rutas
2.9 Modela los campos del formulario
2.10 Valida y explica qué falta
2.11 Elige cuándo mostrar cada error
2.12 Reutiliza el formulario para editar
2.13 Guarda y navega al resultado
2.14 Combina parámetros sin perder filtros
2.15 Cuida la navegación y los formularios en móvil
2.16 Proyecto final: panel con varias pantallas
2.17 Diagnostica problemas frecuentes
2.18 Repaso y glosario

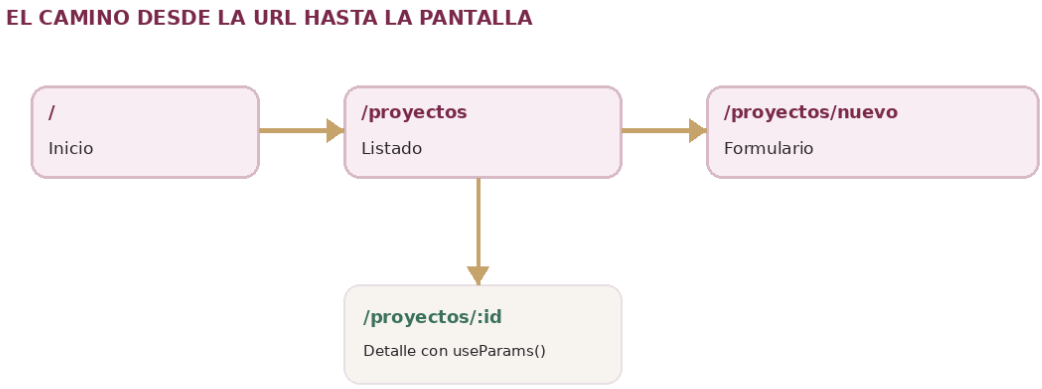
APARTADO 2.1

De una pantalla a varias rutas

Una ruta conecta una dirección del navegador con el componente que debe mostrarse. El usuario puede copiar esa dirección, guardarla y volver directamente a esa pantalla.

Qué cambia para quien usa la app

En el primer libro, la lista, el formulario y el detalle podían estar juntos en una página. Al crecer, una URL por sección hace más fácil orientarse: /proyectos muestra el listado, /agenda muestra el calendario y /proyectos/ID abre una idea concreta.



React Router compara la URL, selecciona la ruta que coincide y dibuja su componente.

Un segmento dinámico como :projectId representa el id del proyecto que se quiere abrir.

Ruta no significa archivo

En este modo, /proyectos/agenda-academica no exige crear una carpeta con ese nombre. React Router lee la URL dentro de la app y elige el componente que corresponde. El servidor de publicación también deberá configurarse para que las rutas de una SPA devuelvan la página principal; volveremos a ese tema en la sección 5.

SPA Una aplicación de una sola página puede cambiar de sección sin pedir un HTML nuevo en cada clic. La dirección sí cambia y el navegador conserva atrás y adelante.

Práctica Escribe tres URLs que tendría sentido ofrecer en tu panel madebm3 y anota qué debería aparecer en cada una.

APARTADO 2.2

Añade React Router al proyecto

React Router se encarga de comparar la dirección actual con las rutas que declaras. La documentación actual incluye un modo declarativo, adecuado para aprender rutas de interfaz.

Instala el paquete

```
npm install react-router
```

Ejecuta el comando en la terminal, dentro de la carpeta del proyecto. Después comprueba que el servidor de Vite sigue arrancando con npm run dev.

Conecta BrowserRouter en main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter><App /></BrowserRouter>
  </StrictMode>
);
```

BrowserRouter escucha la dirección del navegador y habilita los componentes y hooks de React Router dentro de App. Solo hace falta un router que envuelva la app.

Importante para los tutoriales Los ejemplos de React Router v7 se importan desde "react-router". Algunos tutoriales antiguos de v6 usan "react-router-dom". No mezcles imports de versiones distintas: comprueba la versión de tu paquete.

Práctica Guarda el archivo, abre la app y confirma que la página inicial sigue apareciendo. Si aparece un error, revisa primero el nombre del paquete en el import.

APARTADO 2.3

Declara tus primeras rutas

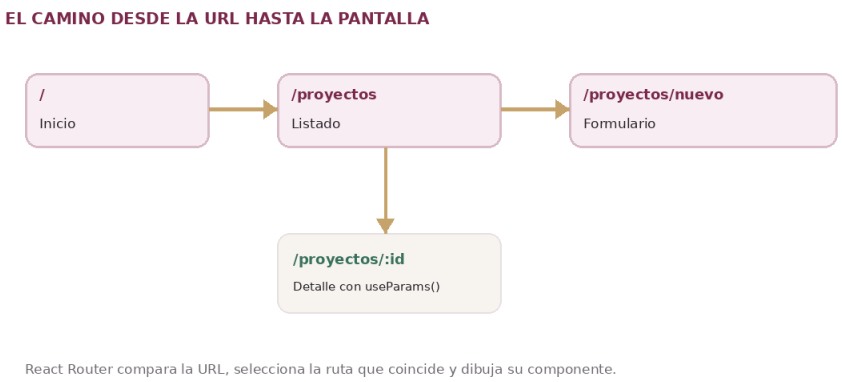
Routes contiene el conjunto de rutas y cada Route relaciona un path con un elemento de React. La ruta index representa la pantalla principal de su grupo.

```
import { Routes, Route } from "react-router";

function App() {
  return (
    <Routes>
      <Route path="/" element={<HomePage />} />
      <Route path="/proyectos" element={<ProjectListPage />} />
      <Route path="/agenda" element={<AgendaPage />} />
      <Route path="*" element={<NotFoundPage />} />
    </Routes>
  );
}
```

Cómo interpretar el árbol

path	La forma de la URL que debe coincidir.
element	El componente que React Router muestra.
index	La pantalla predeterminada dentro de una ruta padre.
*	Una ruta comodín para las direcciones sin coincidencia.



Cada dirección apunta a una pantalla concreta; no mantengas todas las vistas en un único JSX.

Práctica Añade /favoritos y su componente FavoritesPage. Escribe también un mensaje sencillo para NotFoundPage.

APARTADO 2.4

Navega con Link y NavLink

Los enlaces cambian de ruta sin recargar todo el documento. Usa Link para un destino normal y NavLink para una opción de navegación que debe indicar cuál está activa.

```
import { NavLink, Link } from "react-router";

function MainNav() {
  return (
    <nav aria-label="Secciones principales">
      <NavLink to="/" end>Inicio</NavLink>
      <NavLink to="/proyectos">Proyectos</NavLink>
      <NavLink to="/agenda">Agenda</NavLink>
    </nav>
  );
}

<Link to="/proyectos/nuevo">Añadir una idea</Link>
```

NavLink recibe una clase active cuando coincide con la ruta. El atributo end evita que Inicio quede activo también en todas las rutas que comienzan por /. Un enlace debe parecer y comportarse como enlace; reserva button para una acción.



La barra lateral marca Proyectos como sección activa y el botón de nueva idea conduce a su ruta.

Estilo de la sección actual

```
nav a { color: #F5E8ED; text-decoration: none; }
nav a.active { background: #914A66; font-weight: 700; }
nav a:focus-visible { outline: 3px solid #C6A36A; }
```

Accesibilidad Usa palabras que expliquen el destino, deja que los enlaces reciban foco de teclado y mantén visible el indicador de foco.

Práctica Añade NavLink para Ideas y Agenda. Comprueba que cambia la opción activa al entrar directamente en cada URL.

APARTADO 2.5

Comparte la estructura con Outlet

Si las pantallas comparten cabecera, navegación y pie, no copies ese JSX en cada página. Crea una ruta de diseño y coloca Outlet en el sitio donde debe aparecer la ruta hija.

```
import { Outlet } from "react-router";

function AppLayout() {
  return (
    <>
      <Header />
      <main className="page-content"><Outlet /></main>
      <Footer />
    </>
  );
}
```

```
<Routes>
  <Route element={<AppLayout />}>
    <Route index element={<HomePage />} />
    <Route path="proyectos" element={<ProjectListPage />} />
    <Route path="agenda" element={<AgendaPage />} />
  </Route>
</Routes>
```

Outlet es el espacio para el contenido hijo. La cabecera permanece en pantalla mientras cambia el componente de la sección.

Anidamiento de URL Las rutas hijas pueden definirse relativas: dentro de la ruta padre no necesitas comenzar cada path con /. React Router compone el destino con el padre.

Práctica Mueve Header al AppLayout y confirma que aparece en Inicio, Proyectos y Agenda, sin duplicar el componente.

APARTADO 2.6

Abre el detalle con un parámetro

Una ruta dinámica identifica el dato variable con dos puntos. En /proyectos/:projectId, React Router entrega el texto capturado a useParams.

```
<Route path="proyectos/:projectId" element={<ProjectDetailPage projects={projects} />} />

import { useParams, Link } from "react-router";

function ProjectDetailPage({ projects }) {
  const { projectId } = useParams();
  const project = projects.find(item => item.id === projectId);

  if (!project) return <p>No encontramos esa idea.</p>;

  return (
    <article>
      <p><Link to="/proyectos">← Volver a proyectos</Link></p>
      <h1>{project.title}</h1>
      <p>{project.category} · {project.status}</p>
    </article>
  );
}
```

El valor del parámetro es texto. Si tu id es un número, convierte con Number(projectId) antes de comparar. Para ids de texto, como agenda-academica, compara directamente.



Una página de detalle da contexto, permite volver al listado y ofrece acciones relacionadas.

Pensamiento de pantalla Una URL escrita a mano puede apuntar a un id que no existe. Diseña una respuesta visible para ese caso en vez de dar por hecho que siempre hay resultado.

Práctica Haz clic desde una tarjeta y abre /proyectos/<id>. Prueba además una URL con un id inventado.

APARTADO 2.7

Guarda búsqueda y filtros en la URL

Los parámetros después de ? representan opciones de una pantalla. Por ejemplo, /proyectos?estado=Idea puede abrir el listado ya filtrado y se puede compartir como enlace.

```
import { useSearchParams } from "react-router";

function ProjectListPage({ projects }) {
  const [searchParams, setSearchParams] = useSearchParams();
  const status = searchParams.get("estado") ?? "Todos";

  const visibleProjects = projects.filter(project =>
    status === "Todos" || project.status === status
  );

  function chooseStatus(event) {
    const value = event.target.value;
    setSearchParams(value === "Todos" ? {} : { estado: value });
  }
}
```

```
<select value={status} onChange={chooseStatus}>
  <option>Todos</option>
  <option>Idea</option>
  <option>En curso</option>
  <option>Terminado</option>
</select>
```

Al cambiar el select, la dirección se actualiza. Al volver a cargar, el filtro se puede leer de nuevo desde la URL. Si necesitas conservar dos parámetros (por ejemplo búsqueda y estado), actualízalos juntos en una sola llamada a setSearchParams.

Estado local o URL Usa estado local para un detalle efímero de un formulario. Usa la URL cuando convenga compartir, recargar o volver atrás con ese filtro aplicado.

Práctica Añade ?busqueda=collar al listado y muestra solo los proyectos cuyo título contenga ese texto.

APARTADO 2.8

Comparte los proyectos entre rutas

El listado, la ficha y el editor usan la misma colección. Guarda projects cerca de las rutas que la necesitan y pásala como prop. Así cada página consulta una sola fuente de datos.

```
function App() {
  const [projects, setProjects] = useState(starterProjects);
  const addProject = project => setProjects(current => [...current, project]);
  const updateProject = edited => setProjects(current =>
    current.map(item => item.id === edited.id ? edited : item)
  );

  return (
    <Routes>
      <Route element={<AppLayout />}>
        <Route path="proyectos" element={<ProjectListPage projects={projects} />} />
        <Route path="proyectos/nuevo" element={<NewProjectPage onAdd={addProject} />} />
      </Route>
    </Routes>
  );
}
```

Al añadir o editar un proyecto, el padre actualiza projects. React vuelve a representar las rutas que reciben esa prop. El filtro visible se calcula de projects y la URL; no guardes ambos datos como estados independientes.

No hace falta Context todavía Para unas pocas rutas relacionadas, levantar el estado hasta el padre común es sencillo. Más adelante veremos cuándo un contexto o una herramienta adicional mejora el mantenimiento.

Práctica Añade onUpdate y pasa el mismo projects a la ruta de detalle y a la ruta de edición.

APARTADO 2.9

Modela los campos del formulario

Un formulario de proyecto contiene varios valores relacionados. Puedes guardarlos en un objeto de estado y actualizar una sola propiedad por cambio.

```
const [form, setForm] = useState({
  title: "",
  category: "Papelería",
  status: "Idea",
  notes: "",
});

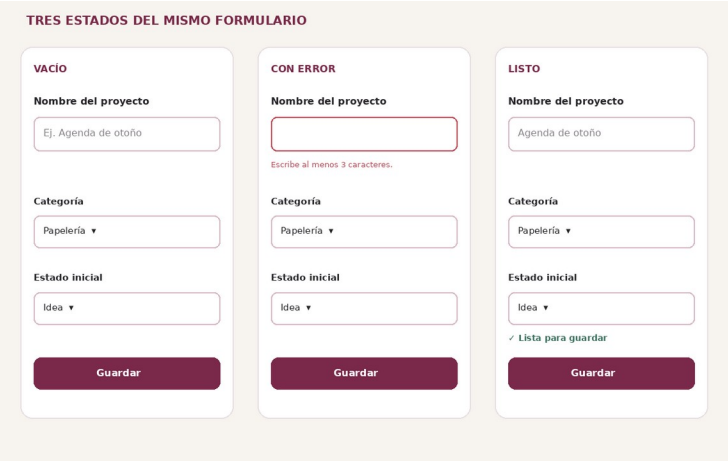
function handleChange(event) {
  const { name, value } = event.target;
  setForm(current => ({ ...current, [name]: value }));
}
```

```
<label htmlFor="title">Nombre del proyecto</label>
<input id="title" name="title" value={form.title} onChange={handleChange} />

<label htmlFor="category">Categoría</label>
<select id="category" name="category" value={form.category} onChange={handleChange}>
  <option>Papelería</option><option>Accesorios</option><option>Decoración</option>
</select>
```

El atributo name indica qué propiedad del objeto debe cambiar. El spread crea un objeto nuevo, conserva las otras propiedades y reemplaza solo la que el usuario acaba de editar.

Una entrada, una etiqueta Cada campo necesita un label conectado mediante htmlFor e id. El placeholder puede dar un ejemplo, pero no reemplaza la etiqueta.



El mismo formulario puede mostrar los estados vacío, con error y listo.

Práctica Añade el campo notes como textarea. Comprueba que cambiar una entrada no borra las otras.

APARTADO 2.10

Valida y explica qué falta

Una validación útil explica cómo corregir el campo. Combina las reglas del navegador con una función de JavaScript cuando necesites mensajes concretos.

```
function validateProject(form) {
  const errors = {};
  const title = form.title.trim();

  if (!title) errors.title = "Escribe un nombre para el proyecto.";
  else if (title.length < 3) errors.title = "Usa al menos 3 caracteres.";
  if (!form.category) errors.category = "Elige una categoría.";

  return errors;
}
```

```
const [errors, setErrors] = useState({});

function handleSubmit(event) {
  event.preventDefault();
  const nextErrors = validateProject(form);
  setErrors(nextErrors);
  if (Object.keys(nextErrors).length > 0) return;
  onSave({ ...form, title: form.title.trim() });
}
```

Conecta el error con su campo

```
<input
  id="title" name="title" value={form.title}
  onChange={handleChange}
  aria-invalid={Boolean(errors.title)}
  aria-describedby={errors.title ? "title-error" : undefined}
/>
{errors.title && <p id="title-error" role="alert">{errors.title}</p>}
```

Muestra el mensaje junto al campo que lo necesita. Usa el color como apoyo, pero no como única señal. Un mensaje que aparece con role="alert" se anuncia a tecnologías de asistencia.

Práctica Añade una regla que rechace nombres de una sola letra y un mensaje que indique cómo solucionarlo.

APARTADO 2.11

Elige cuándo mostrar cada error

Mostrar todos los errores antes de que la persona escriba puede resultar confuso. Una secuencia sencilla valida al enviar y puede volver a revisar el campo que ya se tocó.

```
const [touched, setTouched] = useState({});
const [submitted, setSubmitted] = useState(false);

function handleSubmit(event) {
  event.preventDefault();
  setSubmitted(true);
  // validar, mostrar errores y guardar
}

function handleBlur(event) {
  const { name } = event.target;
  setTouched(current => ({ ...current, [name]: true }));
}

const titleError = (touched.title || submitted)
  ? validateProject(form).title
  : undefined;

<input aria-invalid={Boolean(titleError)} />
{titleError && <p role="alert">{titleError}</p>}
```

Otra estrategia clara es mostrar los errores al enviar y, después, quitarlos conforme se corrigen. Elige una regla coherente para el formulario completo y evita mensajes que aparezcan y desaparezcan sin explicación.

Aprovecha el navegador

```
<input name="title" required minLength={3} maxLength={80} />
<input name="dueDate" type="date" />
<input name="price" type="number" min="0" step="0.01" />
```

required, minLength, type y min dan restricciones útiles al navegador. La validación de React sigue siendo importante para dar mensajes adaptados y para comprobar los datos antes de guardarlos.

Validar no es proteger datos La validación del navegador mejora la experiencia. Si más adelante envías datos a una API, el servidor también debe validarlos.

Práctica Haz que el error del nombre aparezca después de enviar o de salir del campo, no al cargar el formulario vacío.

APARTADO 2.12

Reutiliza el formulario para editar

El formulario puede recibir valores iniciales y una función onSave. En modo creación empieza vacío; en modo edición recibe el proyecto que ya existe.

```
function ProjectForm({ initialProject, onSave }) {
  const [form, setForm] = useState(() => ({
    title: initialProject?.title ?? "",
    category: initialProject?.category ?? "Papelería",
    status: initialProject?.status ?? "Idea",
    notes: initialProject?.notes ?? "",
  }));
  // comparte aquí los campos y la validación
}
```

?? toma el valor de la derecha cuando initialProject no existe.

```
function saveValidForm() {
  onSave({ ...(initialProject ?? {}), ...form });
}
```

Al combinar el proyecto inicial con los campos actuales, la edición conserva su id. En creación no hay proyecto inicial, así que la nueva pantalla añade un id al guardar.

```
function EditProjectPage({ projects, onUpdate }) {
  const { projectId } = useParams();
  const project = projects.find(item => item.id === projectId);
  if (!project) return <NotFoundPage />;
  return <ProjectForm initialProject={project} onSave={onUpdate} />;
}
```

```
<Route path="proyectos/:projectId/editar"
  element={<EditProjectPage projects={projects} onUpdate={updateProject} />}
/>
```

Crear y editar

Nuevo	**initialProject no existe; el título aparece vacío y onSave añade un id.**
Editar	initialProject trae los valores existentes y onSave reemplaza el elemento con el mismo id.

Cambia de proyecto Usa key={project.id} para reiniciar el formulario cuando cambias de proyecto.

Práctica Edita el estado de un proyecto y confirma que vuelves al detalle con los nuevos datos.

APARTADO 2.13

Guarda y navega al resultado

Después de validar, crea el proyecto, actualiza la lista compartida y envía a la persona a la ficha nueva. useNavigate resulta útil cuando la navegación sucede como resultado de un envío.

```
import { useNavigate } from "react-router";

function NewProjectPage({ onAdd }) {
  const navigate = useNavigate();

  function saveProject(form) {
    const project = { ...form, id: crypto.randomUUID() };
    onAdd(project);
    navigate(`/proyectos/${project.id}`);
  }

  return <ProjectForm onSave={saveProject} />;
}
```

En el listado, guarda una clave estable. crypto.randomUUID() genera una clave en navegadores modernos cuando la app corre en un contexto seguro como localhost. Si un navegador no la ofrece, adopta una estrategia de id coherente con tu app.

Estados de envío

```
const [isSaving, setIsSaving] = useState(false);

<button type="submit" disabled={isSaving}>
  {isSaving ? "Guardando…" : "Guardar proyecto"}
</button>
```

Si guardar implica esperar una petición, marca el estado como pendiente y evita dos envíos simultáneos. Al terminar, muestra éxito o un error que permita volver a intentarlo.

El ejemplo sigue siendo local onAdd actualiza el estado en memoria. Al recargar, un proyecto nuevo puede desaparecer si no conectas el almacenamiento local de la sección 1 o una API, que veremos en otro bloque.

Práctica Tras editar un proyecto, navega a su misma dirección de detalle. Añade un enlace para cancelar y volver sin guardar.

APARTADO 2.14

Combina parámetros sin perder filtros

Una URL puede conservar más de un dato: /proyectos?estado=Idea&busqueda=collar. Lee ambos valores y calcula la lista visible desde los proyectos.

```
const [params, setParams] = useSearchParams();
const status = params.get("estado") ?? "Todos";
const query = params.get("busqueda") ?? "";

const visible = projects.filter(project => {
  const matchesStatus = status === "Todos" || project.status === status;
  const matchesText = project.title.toLowerCase().includes(query.toLowerCase());
  return matchesStatus && matchesText;
});
```

```
function updateQuery(event) {
  const next = new URLSearchParams(params);
  const value = event.target.value;
  if (value) next.set("busqueda", value);
  else next.delete("busqueda");
  setParams(next);
}
```

```
<input type="search" value={query} onChange={updateQuery}
  aria-label="Buscar proyectos" />
```

URLSearchParams evita construir la query con concatenaciones manuales y se encarga de codificar caracteres. Actualiza un parámetro conservando los demás cuando ambos filtros deben seguir aplicados.



El panel muestra la selección actual y la URL permite volver a la misma búsqueda.

Práctica Comparte la URL filtrada con una compañera o pégala en otra pestaña. ¿Conserva el filtro esperado?

APARTADO 2.15

Cuida la navegación y los formularios en móvil

Una app con varias rutas también debe ser clara en una pantalla pequeña. El usuario necesita saber dónde está, cómo volver y qué acción es principal.



Una columna de tarjetas y un botón visible mantienen el flujo en móvil.

Comprobación visual

- La opción activa sigue reconocible sin depender solo del color.
- Los enlaces y botones tienen nombres concretos: “Ver agenda académica”, no “aquí”.
- Cada label está conectado con un input o select.
- El mensaje de error aparece cerca del campo y puede leerse con contraste suficiente.
- El foco de teclado se ve; no lo elimines con outline: none sin una alternativa.

En React Router NavLink añade aria-current="page" a la ruta activa. No añadas atributos contradictorios; estiliza el estado activo y comprueba el foco.

Práctica Reduce la ventana del navegador y recorre las rutas con Tab, Enter y Shift+Tab.

APARTADO 2.16

Proyecto final: panel con varias pantallas

Ahora une las rutas con el flujo para crear, leer y editar proyectos. El primer alcance no necesita autenticación, backend ni estado global.

Tabla de rutas

/	**Inicio y resumen de actividad.**
/proyectos	Listado con búsqueda y filtro.
/proyectos/nuevo	Formulario vacío con validación.
/proyectos/:projectId	Detalle de un proyecto.
/proyectos/:projectId/editar	Formulario con datos iniciales.
*	Pantalla de dirección no encontrada.

Orden de construcción

1. Instala el router y envuelve la app con BrowserRouter.
2. Configura rutas y un AppLayout con Outlet.
3. Añade NavLink y prueba cada destino.
4. Conecta el listado y el detalle al mismo estado projects.
5. Crea ProjectForm con etiquetas, validación y mensajes.
6. Reutiliza ProjectForm para edición.
7. Añade búsqueda compartible con search params.
8. Prueba rutas directas, navegación atrás y pantalla estrecha.

APARTADO 2.17

Diagnostica problemas frecuentes

Cuando una vista no aparece, comprueba primero la URL, la ruta y el componente. Después revisa el primer error completo en la consola o terminal.

Pantalla en blanco al abrir URL	**Comprueba que BrowserRouter envuelve App y que la ruta coincide.**
No cambia al pulsar una opción	Usa Link o NavLink, revisa el destino y confirma que la ruta está declarada.
Detalle vacío	Inspecciona useParams y compara el id con el tipo que tiene tu dato.
Filtro se borra al escribir	Copia los search params actuales, cambia solo la clave necesaria y actualiza una vez.
Formulario siempre da error	Revisa el name de los inputs, form.title y el objeto que devuelve validateProject.
Editar muestra los datos viejos	Confirma el id, las props initialProject y si el componente necesita una key estable.

Lista de pruebas manuales

- ☐ Abre cada ruta escribiéndola en el navegador.
- ☐ Recarga una página de detalle y observa qué datos se conservan.
- ☐ Prueba una URL desconocida y un id inexistente.
- ☐ Envía el formulario vacío y revisa el foco, los mensajes y aria-invalid.
- ☐ Envía datos válidos, edita la ficha y vuelve al listado.
- ☐ Activa filtro y búsqueda; usa atrás y adelante del navegador.

Bitácora Anota un error de ruta o de formulario que te haya aparecido y la comprobación que te ayudó a resolverlo.

APARTADO 2.18

Repaso y glosario

Comprueba qué puedes explicar sin mirar el código. Si algo cuesta, vuelve al apartado y reconstruye el ejemplo dentro de tu propia app.

- ☐ Sé explicar la diferencia entre declarar una ruta y dibujar un enlace.
- ☐ Puedo instalar React Router v7 y envolver App con BrowserRouter.
- ☐ Puedo crear rutas normales, anidadas y dinámicas.
- ☐ Sé cuándo usar Link, NavLink y useNavigate.
- ☐ Puedo leer ids con useParams y filtros con useSearchParams.
- ☐ Sé compartir projects entre listado, detalle y formulario.
- ☐ Puedo validar, editar y explicar errores de un formulario.
- ☐ Compruebo teclado, foco, labels y pantalla pequeña.

Glosario

Ruta — Regla que relaciona una dirección con un componente.

SPA — Aplicación que cambia de vista en el navegador sin solicitar un HTML nuevo en cada clic.

Outlet — Lugar donde aparece el contenido de una ruta hija.

Parámetro dinámico — Segmento como :projectId que se lee con useParams.

Search params — Valores de la query string, tras ?, por ejemplo estado=Idea.

Validación — Reglas que explican si los campos pueden guardarse y qué corregir.

sección 3 El siguiente bloque puede añadir una biblioteca de interfaz, iconos y decisiones de diseño, comparando qué resuelve cada opción antes de instalarla.

SECCIÓN 3

Interfaz y librerías

Da coherencia visual a madebm3 con Material UI, un tema y componentes reutilizables.

Al terminar esta sección

Al terminar podrás instalar MUI, componer tarjetas y campos, y adaptar la interfaz a pantallas pequeñas.

Apartados

3.1 Qué es una librería de interfaz
3.2 Conoce la estructura de MUI
3.3 Monta tu primer componente MUI
3.4 Define los colores de madebm3 con un tema
3.5 Personaliza un componente con sx
3.6 Construye una tarjeta reutilizable
3.7 Usa Stack y Grid para distribuir contenido
3.8 Estiliza títulos y jerarquía con Typography
3.9 Construye campos y selects
3.10 Convierte un ejemplo visual en un formulario real
3.11 Añade iconos sin sustituir las palabras
3.12 Muestra estados de carga, vacío y error
3.13 Conserva React Router junto a MUI
3.14 Haz que la aplicación se adapte a móvil
3.15 Organiza los componentes del proyecto
3.16 Proyecto final: renueva el panel madebm3
3.17 Resuelve fallos frecuentes
3.18 Repaso y glosario

APARTADO 3.1

Qué es una librería de interfaz

Una librería de interfaz reúne componentes listos para usar: botones, campos, menús, tarjetas, cuadros de diálogo y piezas de diseño. Cada componente ya incorpora comportamiento y estilos que puedes configurar.

Librería, marco CSS o componente propio

Librería de componentes	Importas componentes React como Button, Card o TextField y los configuras con props.
Marco de CSS	Añades clases a tu HTML o JSX; el marco aporta estilos y convenciones.
Componente propio	Defines tú mismo el JSX, los estilos y el comportamiento que necesita tu producto.

MUI es una librería React que implementa componentes de Material Design y permite personalizar la paleta, tipografía, forma y otros detalles. Instalarla no convierte la app en un diseño terminado: todavía eliges jerarquía, contenido, espacios y estados.

Cuándo compensa usarla

- Tienes varias pantallas con los mismos patrones de botones, campos y tarjetas.
- Necesitas construir una interfaz funcional sin inventar cada control desde cero.
- Te viene bien que el tema reúna colores, espacios y tipografía.
- El equipo puede aceptar el estilo de base o invertir tiempo en personalizarlo.

Decisión práctica Para una tarjeta única puede bastar CSS propio. Si cada pantalla empieza a repetir las mismas reglas y controles, una librería puede reducir trabajo y variaciones accidentales.

Práctica Anota tres controles repetidos de tu app madebm3 y qué apariencia o comportamiento deberían compartir.

APARTADO 3.2

Conoce la estructura de MUI

El paquete principal proporciona los componentes. Emotion es el motor de estilos predeterminado. Los iconos oficiales se instalan por separado, así que puedes añadirlos cuando realmente los uses.

Instala solo lo que usarás

```
npm install @mui/material @emotion/react @emotion/styled
```

Ejecuta el comando desde la carpeta del proyecto. React y React DOM ya deberían estar instalados en el proyecto Vite. Para los iconos de Material Symbols, añade el paquete cuando vayas a emplearlos:

```
npm install @mui/icons-material
```

Importa lo que necesitas

```
import Button from "@mui/material/Button";
import Card from "@mui/material/Card";
import CardContent from "@mui/material/CardContent";
import Typography from "@mui/material/Typography";
```

Cada import identifica claramente el componente. En los ejemplos de este manual se usa el paquete actual @mui/material. Tutoriales antiguos pueden usar nombres, rutas de importación o APIs que han cambiado.

Comprueba la instalación Arranca Vite con npm run dev. Si el proyecto no encuentra un módulo, revisa que corriste npm install en la carpeta que contiene package.json y que el nombre del paquete está escrito igual en el import.

Práctica Instala MUI, importa Button en una pantalla y muestra un botón con el texto “Nueva idea”.

APARTADO 3.3

Monta tu primer componente MUI

Un componente MUI se escribe como JSX de React. Las props determinan variante, color, tamaño y otras características. Empieza con pocas props y observa qué cambia cada una.

```
import Button from "@mui/material/Button";

function AddIdeaButton() {
  return (
    <Button variant="contained" color="primary">
      Nueva idea
    </Button>
  );
}
```

Tres variantes para tres niveles de énfasis

contained	Acción principal, por ejemplo guardar una nueva idea.
outlined	Acción secundaria con presencia clara, por ejemplo cancelar.
text	Acción de menor énfasis, por ejemplo abrir ayuda dentro de una tarjeta.

```
<Button variant="contained">Guardar idea</Button>
<Button variant="outlined">Cancelar</Button>
<Button variant="text">Ver ayuda</Button>
```

El botón está pensado para ejecutar una acción. Para navegar a otra página usa un enlace. Mantener separados esos propósitos ayuda al teclado, a lectores de pantalla y a las expectativas de quien usa la app.

Práctica Crea una acción principal y una secundaria para el formulario. Decide cuál debe llamar más la atención y explícalo en una frase.

APARTADO 3.4

Define los colores de madebm3 con un tema

Un tema concentra decisiones que se repiten: colores, tipografía, formas, espaciado y estados. Si el color principal cambia, actualizas el tema en un lugar y sus componentes lo heredan.

```
import { createTheme } from "@mui/material/styles";

export const theme = createTheme({
  palette: {
    primary: { main: "#7A294A" },
    secondary: { main: "#C6A36A" },
    background: { default: "#F7F4EF", paper: "#FFFFFF" },
    text: { primary: "#27242A", secondary: "#706B72" },
  },
  shape: { borderRadius: 12 },
});
```

Guarda este ejemplo en src/theme.js. Los nombres primary y secondary dan significado a los colores: es más fácil mantener un tema que repetir códigos hexadecimales distintos en cada pantalla.

Activa el tema una sola vez

```
import { ThemeProvider } from "@mui/material/styles";
import CssBaseline from "@mui/material/CssBaseline";
import { theme } from "./theme";

root.render(
  <ThemeProvider theme={theme}>
    <CssBaseline />
    <App />
  </ThemeProvider>
);
```

ThemeProvider comparte el tema con los componentes descendientes. CssBaseline aplica una base global coherente. Si ya envuelves la app con BrowserRouter de la sección 2, coloca ambos proveedores de forma que BrowserRouter y MUI sigan envolviendo la app completa; no reemplaces uno por el otro.

Práctica Cambia el color primario del tema y observa qué botones lo heredan. Anota qué elemento de tu marca no debería usar ese color como fondo.

APARTADO 3.5

Personaliza un componente con sx

La prop sx aplica estilos puntuales con acceso a valores del tema. Es útil para pequeños ajustes locales; deja los patrones compartidos en el tema o en un componente reutilizable.

```
<Button
  variant="contained"
  sx={{ borderRadius: 3, px: 2.5, textTransform: "none" }}
>
  Guardar idea
</Button>
```

En sx, px representa padding horizontal. Las unidades numéricas de ciertas propiedades usan el sistema de espacios de MUI. Puedes usar valores directos de CSS cuando necesites una medida concreta.

```
<Card sx={{ borderRadius: 3, boxShadow: 1 }}>
  <CardContent>
    <Typography variant="h6">Agenda académica</Typography>
  </CardContent>
</Card>
```

Evita dos sistemas que se contradicen

Si una regla global de tu index.css fuerza todos los botones, puede competir con MUI. Retira o limita los estilos antiguos que ya no corresponden. No hace falta borrar todos tus estilos: CSS sigue siendo útil para el diseño general, imágenes, animaciones sencillas y piezas propias.

Regla simple Usa el tema para decisiones compartidas, sx para un ajuste pequeño y CSS propio cuando la composición visual lo requiera. Evita copiar diez veces el mismo objeto sx.

Práctica Ajusta el radio de una tarjeta con sx. Cambia también un estilo del tema y compara qué opción te conviene para repetirlo.

APARTADO 3.6

Construye una tarjeta reutilizable

Una tarjeta de proyecto debe representar datos, no tener nombres y contenidos fijados dentro. Recibe un proyecto por prop y separa los datos del diseño.

```
import Card from "@mui/material/Card";
import CardContent from "@mui/material/CardContent";
import Chip from "@mui/material/Chip";
import Stack from "@mui/material/Stack";
import Typography from "@mui/material/Typography";

function ProjectCard({ project }) {
  return (
    <Card>
      <CardContent>
        <Stack spacing={1.5}>
          <Typography variant="h6">{project.title}</Typography>
          <Typography color="text.secondary">{project.category}</Typography>
          <Chip label={project.status} size="small" />
        </Stack>
      </CardContent>
    </Card>
  );
}
```

Puedes pasar el mismo componente a la lista con projects.map. Conserva key en el elemento devuelto por cada iteración, como en la sección 1. La librería visual no cambia las reglas de listas de React.

```
{projects.map(project => (
  <ProjectCard key={project.id} project={project} />
))}
```

Práctica Añade una propiedad dueDate o notes a un proyecto y decide si debe aparecer siempre, solo en el detalle o en un tooltip.

APARTADO 3.7

Usa Stack y Grid para distribuir contenido

Stack organiza sus elementos en una dirección. Grid organiza contenido en columnas que pueden cambiar en distintos tamaños. Elige cada uno según la forma del problema, no por decorar el JSX.

Una dimensión con Stack

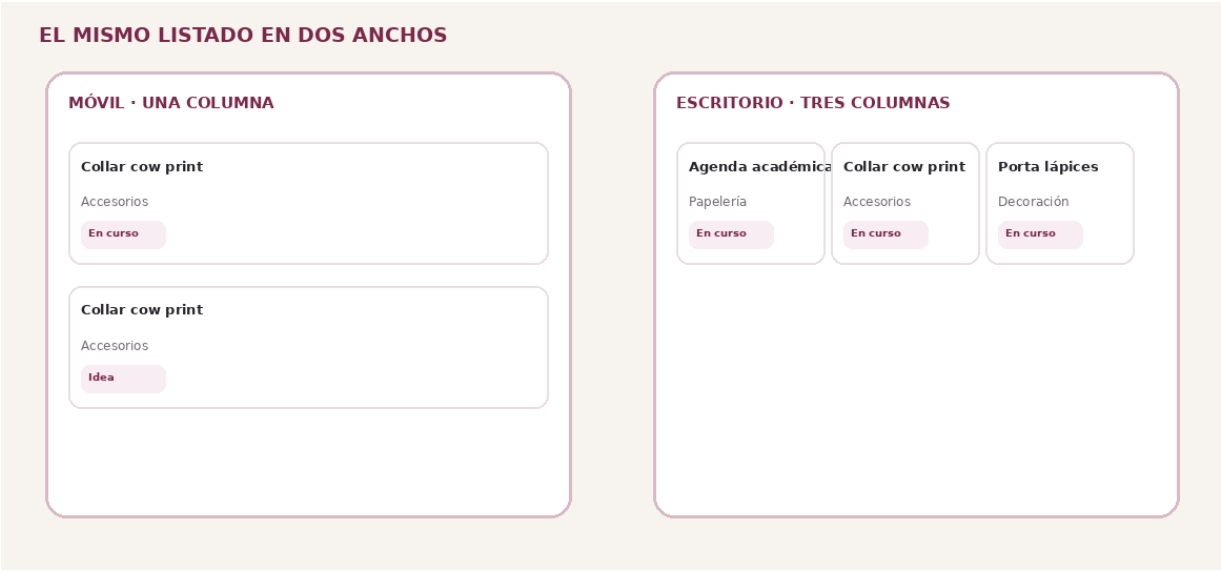
```
<Stack direction="row" spacing={2} alignItems="center">
  <Typography variant="h4">Tus proyectos</Typography>
  <Button variant="contained">Nueva idea</Button>
</Stack>
```

Tarjetas en una cuadrícula adaptable

```
import Grid from "@mui/material/Grid";

<Grid container spacing={2}>
  {projects.map(project => (
    <Grid key={project.id} size={{ xs: 12, sm: 6, md: 4 }}>
      <ProjectCard project={project} />
    </Grid>
  ))}
</Grid>
```

El valor xs ocupa las 12 columnas en el ancho pequeño, sm usa seis columnas a partir del punto de corte y md usa cuatro. Así caben una, dos o tres tarjetas según el ancho disponible. El Grid actual usa size; algunos tutoriales antiguos hablan de item y xs en el hijo.



Una tarjeta ocupa el ancho disponible en móvil y se reparte en columnas en pantallas más amplias.

Práctica Ajusta el punto de corte para que las tarjetas pasen a dos columnas antes. Comprueba el resultado estrechando poco a poco la ventana.

APARTADO 3.8

Estiliza títulos y jerarquía con Typography

Typography traduce una intención de contenido a una escala visual. Usa el nivel de título según la estructura y la prop variant para escoger su presentación.

```
<Typography variant="h1" component="h1">
  Mis proyectos
</Typography>
<Typography variant="body1" color="text.secondary">
  Ideas para agendas, accesorios y piezas hechas a mano.
</Typography>
```

variant controla el estilo visual. component controla la etiqueta semántica HTML que se genera. Puedes mostrar el estilo h4 y conservar el nivel de título apropiado para la estructura de la página:

```
<Typography variant="h4" component="h1">Tus proyectos</Typography>
```

Jerarquía antes que tamaño

Una página debe tener un título principal que describa su contenido. Los subtítulos organizan secciones. No elijas una etiqueta por cómo se ve: usa los estilos de MUI para que una jerarquía semántica correcta también se vea como deseas.

Práctica Revisa Inicio, Proyectos y Detalle. Anota el h1 de cada vista y comprueba que expresa el destino de la página.

APARTADO 3.9

Construye campos y selects

MUI ofrece controles visuales listos, pero los formularios siguen usando las reglas de HTML y React. Cada campo necesita una etiqueta comprensible y un valor conectado al estado si el formulario es controlado.

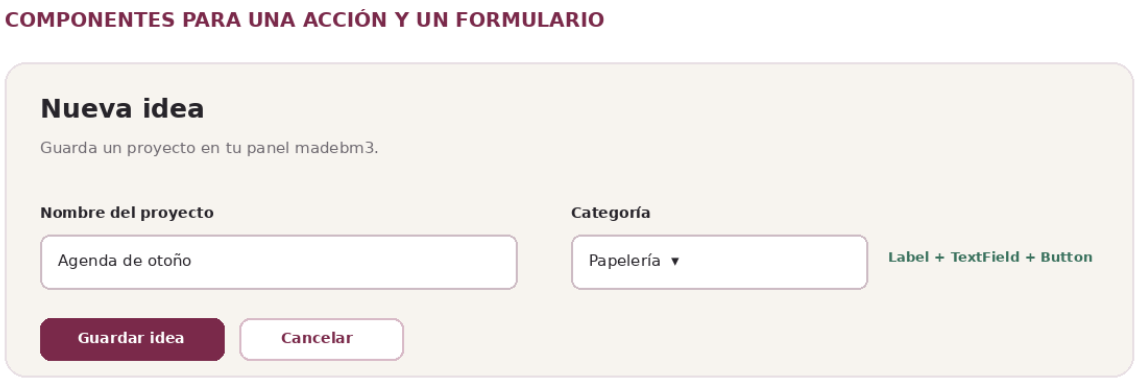
```
import TextField from "@mui/material/TextField";

<TextField
  label="Nombre del proyecto"
  name="title"
  value={form.title}
  onChange={handleChange}
  fullWidth
  required
/>
```

Añade error y ayuda

```
<TextField
  label="Nombre del proyecto"
  value={form.title}
  onChange={handleChange}
  error={Boolean(errors.title)}
  helperText={errors.title || "Por ejemplo, Agenda de otoño"}
/>
```

Cuando el valor no cumple la regla, activa error y presenta texto útil con helperText. Evita mostrar solo un borde rojo: explica qué campo falla y qué puede hacer la persona para resolverlo.



Un formulario combina etiquetas, campos y botones; el texto mantiene claro el propósito de cada control.

Práctica Conecta un TextField controlado al estado del formulario de la sección 2 y añade una regla para título vacío.

APARTADO 3.10

Convierte un ejemplo visual en un formulario real

Un formulario completo recoge datos, valida y llama a la función que ya existe. MUI cambia la presentación del control, no la lógica de submit ni el manejo de errores.

```
import { useState } from "react";
import Box from "@mui/material/Box";
import Button from "@mui/material/Button";
import TextField from "@mui/material/TextField";

function ProjectForm({ onSave }) {
  const [title, setTitle] = useState("");
  const [error, setError] = useState("");

  function handleSubmit(event) {
    event.preventDefault();
    if (title.trim().length < 3) {
      setError("Escribe al menos 3 caracteres.");
      return;
    }
    onSave({ title: title.trim() });
  }

  return (
    <Box component="form" onSubmit={handleSubmit}>
      <TextField label="Nombre del proyecto" value={title}
        onChange={event => setTitle(event.target.value)}
        error={Boolean(error)} helperText={error} />
      <Button type="submit" variant="contained">Guardar idea</Button>
    </Box>
  );
}
```

Este fragmento muestra el flujo esencial. En tu app, integra la categoría, el estado inicial y la función onAdd que ya definiste en la sección 2. Si el formulario ya funciona, cambia un control cada vez para encontrar errores con facilidad.

Separa presentación y reglas

ProjectForm es responsable de leer y presentar campos. La función onSave del padre es responsable de añadir el proyecto. Las reglas de validación deberían seguir cerca del formulario o en una función compartida, pero no mezcladas con la paleta del tema.

Práctica Cambia el botón de submit y el TextField de la sección 2 por los de MUI. Conserva tus reglas y verifica envío vacío, envío válido y foco de teclado.

APARTADO 3.11

Añade iconos sin sustituir las palabras

Los iconos ayudan a reconocer acciones habituales, pero deben acompañarse de una etiqueta textual o un nombre accesible. Instala el paquete de iconos solo si tu interfaz los utiliza.

```
import AddIcon from "@mui/icons-material/Add";
import Button from "@mui/material/Button";

<Button variant="contained" startIcon={<AddIcon />}>
  Nueva idea
</Button>
```

Si un icono no aporta información nueva, no lo añadas. Si el botón solo contiene un icono, utiliza IconButton y proporciona aria-label descriptivo.

```
import IconButton from "@mui/material/IconButton";
import DeleteOutlineIcon from "@mui/icons-material/DeleteOutline";

<IconButton aria-label="Eliminar Collar cow print">
  <DeleteOutlineIcon />
</IconButton>
```

Piensa en teclado y lector de pantalla

- Mantén un texto visible junto a las acciones importantes.
- Añade aria-label a un botón que no tiene texto visible.
- No uses un icono rojo como única señal de que algo se va a eliminar.
- Si la acción es destructiva, explica su consecuencia y pide confirmación cuando haga falta.

Práctica Añade un icono al botón Nueva idea. Después crea un botón solo de icono para editar una tarjeta y escribe su aria-label con el título del proyecto.

APARTADO 3.12

Muestra estados de carga, vacío y error

Las pantallas reales no tienen siempre una lista completa. Una interfaz debe explicar qué ocurre cuando aún carga, cuando no hay resultados o cuando algo falla.

Lista vacía y búsqueda sin coincidencias

```
{projects.length === 0 ? (
  <Alert severity="info">Todavía no has añadido ideas.</Alert>
) : visibleProjects.length === 0 ? (
  <Alert severity="info">No hay proyectos con ese filtro.</Alert>
) : (
  <ProjectGrid projects={visibleProjects} />
)}
```

Error y carga

```
{isLoading && <CircularProgress aria-label="Cargando proyectos" />}
{error && <Alert severity="error">{error}</Alert>}
```

Estos estados se conectarán a una API en la sección 4. Por ahora, crea una lista vacía y una búsqueda sin resultados para comprobar que cada mensaje tiene sentido.

Texto útil “Ha ocurrido un error” es poco accionable. Cuando sepas qué ha fallado, explica el siguiente paso: volver a intentar, revisar un dato o regresar al listado.

Práctica Diseña tres mensajes breves para lista vacía, búsqueda sin coincidencias y error al cargar. Indica una acción adecuada en el estado de error.

APARTADO 3.13

Conserva React Router junto a MUI

Los componentes visuales no reemplazan la navegación de la sección 2. Puedes hacer que Button dibuje un Link de React Router mediante la prop component y pasarle el destino como to.

```
import Button from "@mui/material/Button";
import { Link } from "react-router";

<Button component={Link} to="/proyectos/nuevo" variant="contained">
  Nueva idea
</Button>
```

El componente se ve como un botón, pero se comporta como enlace: el navegador puede mostrar su destino, abrirlo en una pestaña y seguir el flujo de teclado esperado. Usa Button con onClick para acciones que ocurren en la misma vista.

Atención a los enlaces dentro de tarjetas

```
<Button component={Link} to={`/proyectos/${project.id}`}
  variant="text">
  Ver {project.title}
</Button>
```

Mantén el destino descriptivo. Si toda la tarjeta se hace clicable, revisa que no contenga botones o enlaces anidados. Un enlace no debe envolver otros elementos interactivos.

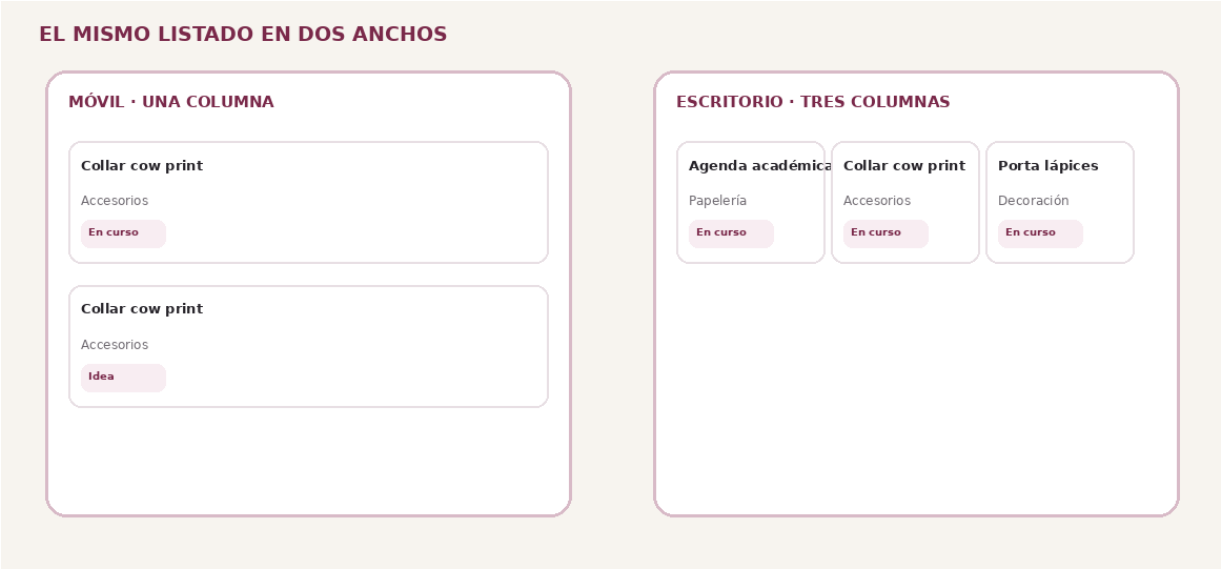
Práctica Convierte el enlace de detalle de la sección 2 en un botón de texto MUI y confirma que la dirección cambia sin recargar toda la página.

APARTADO 3.14

Haz que la aplicación se adapte a móvil

La distribución debe ajustarse a su espacio, no encoger una página de escritorio hasta volverla ilegible. Grid adapta columnas y Stack puede cambiar dirección y separación según breakpoints.

```
<Stack
  direction={{ xs: "column", sm: "row" }}
  spacing={{ xs: 1, sm: 2 }}
  alignItems={{ xs: "stretch", sm: "center" }}
>
  <Typography variant="h4">Tus proyectos</Typography>
  <Button variant="contained">Nueva idea</Button>
</Stack>
```



Compara el ancho estrecho con la distribución de escritorio.

Revisión manual rápida

- ☐ No aparece scroll horizontal por culpa de una tarjeta ancha.
- ☐ Los botones principales caben y se pueden pulsar con un dedo.
- ☐ El texto no se corta en títulos largos.
- ☐ La búsqueda no ocupa más espacio del necesario.
- ☐ La navegación se puede recorrer con teclado.

Práctica Prueba tu app en ancho móvil y escritorio. Cambia un título de proyecto por uno largo para comprobar que las tarjetas siguen siendo legibles.

APARTADO 3.15

Organiza los componentes del proyecto

A medida que importas más componentes, conviene reunir las piezas propias por responsabilidad. No crees una carpeta por cada línea; separa cuando un componente o archivo tenga un propósito claro.

```
src/
  App.jsx
  main.jsx
  theme.js
  components/
    ProjectCard.jsx
    ProjectGrid.jsx
    ProjectForm.jsx
    MainLayout.jsx
  pages/
    HomePage.jsx
    ProjectListPage.jsx
    ProjectDetailPage.jsx
```

Cada carpeta puede empezar más pequeña. Guarda theme.js junto a la configuración de la app. Una tarjeta reutilizable vive en components; una pantalla que corresponde a una ruta vive en pages. No hace falta cambiar toda la estructura del proyecto antes de aprender los componentes.

Una señal de que conviene extraer un componente

- Repites el mismo bloque de JSX en dos o más vistas.
- El bloque tiene un propósito o comportamiento reconocible.
- Sus props permiten contar con claridad qué datos necesita.
- Puedes probarlo de forma independiente dentro de una página.

Práctica Mueve ProjectCard a src/components/ProjectCard.jsx, impórtalo en la lista y confirma que cada tarjeta conserva el mismo contenido.

APARTADO 3.16

Proyecto final: renueva el panel madebm3

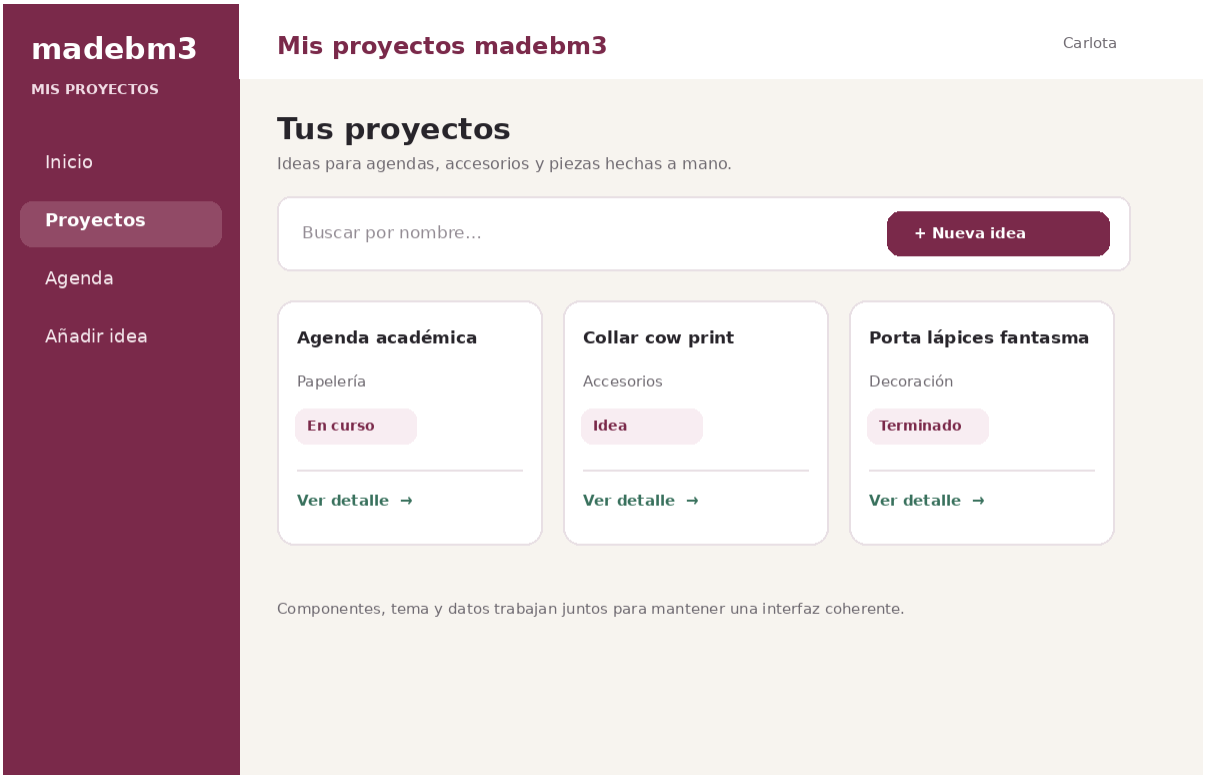
Integra la librería sobre tu app existente. Cambia de manera incremental: instala, activa tema, actualiza la pantalla principal, actualiza formularios, adapta tarjetas y comprueba la navegación.

Lista de pantallas

Inicio	Resumen breve y acceso claro al listado.
Proyectos	Búsqueda, filtro, estado vacío y tarjetas adaptables.
Detalle	Datos del proyecto y enlaces de acción.
Nueva idea	Formulario con etiquetas y errores visibles.
Editar idea	Datos iniciales, botón guardar y salida segura.

Orden de construcción

- Haz una copia del proyecto antes de tocar la interfaz.
- Instala Material UI; confirma que Vite arranca.
- Crea theme.js e incorpora ThemeProvider y CssBaseline.
- Actualiza los títulos y acciones con Typography y Button.
- Construye ProjectCard y renderiza el array con map y key.
- Distribuye las tarjetas con Grid y valores responsive.
- Cambia un campo del formulario por TextField y prueba validación.
- Integra iconos solo donde clarifican la acción.
- Revisa las rutas, el teclado y el ancho de móvil.



Resultado de referencia: listado, filtros, acción principal y tarjetas coherentes.

Entrega de tu práctica Anota tres componentes MUI que ya emplea la app y una pieza visual propia que decidiste conservar.

APARTADO 3.17

Resuelve fallos frecuentes

Si la interfaz deja de compilar después de instalar MUI, revisa el mensaje exacto de la terminal y el import que aparece en la primera línea del error.

Cannot find module @mui/material	Comprueba instalación, carpeta actual y nombre del paquete.
El tema no cambia un botón	Asegúrate de que ThemeProvider envuelve ese componente y que color usa primary o secondary.
El campo no actualiza el formulario	Revisa value, onChange y el estado de React; MUI no sustituye el estado.
Grid no reconoce item o xs	Consulta el API de Grid de la versión instalada; el ejemplo actual utiliza size.
Botón de navegación recarga	Comprueba que component apunta al Link de React Router y que recibe to.
CSS anterior anula el aspecto	Busca reglas globales para button, input o h1 y limita su alcance.

Comprobaciones manuales

- ☐ Abre inicio, listado, detalle y formulario.
- ☐ Escribe texto en búsqueda y en un TextField.
- ☐ Envía formulario vacío y formulario válido.
- ☐ Comprueba mensajes de error, lista vacía y filtro sin resultados.
- ☐ Activa enlaces y botones con teclado.
- ☐ Revisa los colores y la distribución en pantalla estrecha.

Bitácora Anota el primer error que resolviste al instalar una librería y el indicio que te ayudó a encontrarlo.

APARTADO 3.18

Repaso y glosario

La librería aporta controles, pero tú conservas las responsabilidades de React: componentes, props, estado, rutas, datos y decisiones de producto.

- ☐ Puedo explicar qué problema resuelve una biblioteca de interfaz.
- ☐ Sé instalar MUI y distinguir sus imports actuales de tutoriales antiguos.
- ☐ Puedo definir un tema y activar ThemeProvider.
- ☐ Sé cuándo usar sx y cuándo un componente propio.
- ☐ Puedo crear tarjetas reutilizables con props.
- ☐ Puedo distribuir tarjetas con Grid y ajustar breakpoints.
- ☐ Los campos conservan etiquetas, estado y validación accesibles.
- ☐ Sé enlazar un botón visual con React Router sin perder semántica.
- ☐ Compruebo estados vacíos, carga, errores, foco y ancho móvil.

Glosario

Biblioteca de componentes — Paquete de piezas React reutilizables con estilos y comportamiento.

Tema — Configuración común de colores, tipografía, forma y otros valores.

ThemeProvider — Proveedor que hace disponible el tema a los componentes descendientes.

sx — Prop de MUI para ajustar estilos locales con acceso a valores del tema.

Breakpoint — Punto de corte usado para cambiar una distribución según el ancho.

Grid — Componente de diseño de MUI para distribuir elementos en columnas adaptables.

Siguiente sección. El bloque de datos y servicios conecta el panel a una API, maneja respuestas y permite guardar cambios.

SECCIÓN 4

Datos y servicios

Conecta las pantallas a una API local de práctica y aprende a guardar datos con HTTP.

Al terminar esta sección

Al terminar podrás separar servicios, manejar carga y errores y practicar operaciones CRUD.

Apartados

4.1 Qué es una API
4.2 Levanta la API local
4.3 Lee una respuesta con fetch
4.4 Separa el servicio de las pantallas
4.5 Carga datos cuando se abre el listado
4.6 Muestra carga, error y lista vacía
4.7 Crea con POST
4.8 Edita con PATCH
4.9 Borra con DELETE
4.10 Vuelve a cargar sin copiar el efecto
4.11 Configura la dirección de la API
4.12 Filtra resultados con parámetros
4.13 Evita que responda una petición antigua
4.14 Muestra el progreso al guardar
4.15 Diagnostica peticiones con Network
4.16 Proyecto final: conecta el panel
4.17 Resuelve problemas frecuentes
4.18 Repaso y siguientes pasos

APARTADO 4.1

Qué es una API

Una API define cómo puede pedir o modificar datos un programa. El navegador envía una petición HTTP a una dirección y recibe una respuesta, normalmente en JSON.

GET /projects	Devuelve la lista.
GET /projects/1	Devuelve el proyecto 1.
POST /projects	Crea un proyecto.
PATCH /projects/1	Cambia campos del proyecto 1.
DELETE /projects/1	Borra el proyecto 1.

La URL identifica el recurso y el método describe la acción. Espera la respuesta del servidor antes de anunciar que el cambio se guardó.

Cliente y servidor React corre en el navegador y es el cliente. JSON Server será un servidor local. En una app publicada, la API viviría en un servicio accesible por internet.

Práctica Escribe la URL para listar proyectos y la que abriría uno concreto. ¿Qué parte cambia si buscas otro proyecto?

APARTADO 4.2

Levanta la API local

JSON Server convierte un archivo JSON en rutas de prueba. No es un backend de producción: servirá para practicar HTTP en tu ordenador.

Instala la versión estable

```
npm install --save-dev json-server@0.17.4
```

Crea db.json en la raíz del proyecto, junto a package.json:

```
{
  "projects": [
    { "id": 1, "title": "Agenda académica",
      "category": "Papelería", "status": "En curso" },
    { "id": 2, "title": "Collar cow print",
      "category": "Accesorios", "status": "Idea" }
  ]
}
```

Añade un comando de inicio

```
"scripts": {
  "dev": "vite",
  "api": "json-server --watch db.json --port 3000"
}
```

Conserva los scripts que ya existen y añade api dentro del objeto. Abre dos terminales: ejecuta npm run api en una y npm run dev en la otra. La API local debe responder en http://localhost:3000.

Práctica Abre http://localhost:3000/projects. Debes ver los dos proyectos ficticios.

APARTADO 4.3

Lee una respuesta con fetch

fetch devuelve una promesa. La respuesta HTTP llega dentro de esa promesa; response.json() convierte el cuerpo JSON en un valor de JavaScript.

```
const response = await fetch("http://localhost:3000/projects");

if (!response.ok) {
  throw new Error("No se pudieron cargar los proyectos.");
}

const projects = await response.json();
console.log(projects);
```

fetch no rechaza la promesa automáticamente por un 404 o 500. Revisa response.ok; los errores de red sí pueden rechazarla.

response.status	Código de estado HTTP, por ejemplo 200 o 404.
response.ok	Verdadero para respuestas exitosas.
response.json()	Convierte el cuerpo JSON.
await	Espera el resultado de la promesa.

Práctica Prueba /projects/1 y después /projects/999. Observa la respuesta y su código HTTP.

APARTADO 4.4

Separa el servicio de las pantallas

Centraliza los endpoints y las reglas para interpretar respuestas en un módulo. La pantalla se queda con la interfaz y llama a funciones con nombres claros.

```
const API_URL = "http://localhost:3000";

export async function getProjects({ signal } = {}) {
  const response = await fetch(`${API_URL}/projects`, { signal });
  if (!response.ok) throw new Error("No se pudieron cargar las ideas.");
  return response.json();
}

export async function getProject(id, { signal } = {}) {
  const response = await fetch(`${API_URL}/projects/${id}`, { signal });
  if (!response.ok) throw new Error("No se encontró el proyecto.");
  return response.json();
}
```

Crea src/services/projectsApi.js. El parámetro signal permite cancelar una lectura que la pantalla ya no necesita. Si cambia la dirección de la API, modificas este módulo o la configuración.

Práctica Importa getProjects y muestra cuántas ideas devuelve. Aún no cambies el formulario ni las tarjetas.

APARTADO 4.5

Carga datos cuando se abre el listado

La pantalla necesita solicitar proyectos y actualizar el estado cuando llega la respuesta. Un efecto puede sincronizarla con este servicio externo.

```
const [projects, setProjects] = useState([]);
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState("");

useEffect(() => {
  const controller = new AbortController();
  async function load() {
    setIsLoading(true); setError("");
    try {
      const data = await getProjects({ signal: controller.signal });
      setProjects(data);
    } catch (err) {
      if (err.name !== "AbortError") setError(err.message);
    } finally {
      if (!controller.signal.aborted) setIsLoading(false);
    }
  }
  load();
  return () => controller.abort();
}, []);
```

Si la petición depende de un id que puede cambiar, inclúyelo en las dependencias. El cleanup cancela el trabajo cuando la vista ya no necesita esa respuesta.

StrictMode En desarrollo React puede montar, limpiar y volver a ejecutar efectos para encontrar fallos de limpieza. No confundas una petición cancelada en ese ciclo con un error de servidor.

Práctica Apaga JSON Server y abre el listado. Muestra un error; al volver a iniciar el servidor, reintenta la carga.

APARTADO 4.6

Muestra carga, error y lista vacía

Mientras espera, la pantalla debe decir que está trabajando. Si falla, explica el siguiente paso. Una respuesta correcta con cero proyectos merece un mensaje distinto.

```
{isLoading ? (
  <CircularProgress aria-label="Cargando proyectos" />
) : error ? (
  <Alert severity="error">{error}
    <Button onClick={reload}>Volver a intentar</Button>
  </Alert>
) : projects.length === 0 ? (
  <Alert severity="info">Todavía no has añadido ideas.</Alert>
) : (
  <ProjectGrid projects={visibleProjects} />
)}
```



La interfaz distingue espera, error y ausencia de proyectos.

```
const [reloadKey, setReloadKey] = useState(0);
const reload = () => setReloadKey(key => key + 1);
```

Añade reloadKey a las dependencias del efecto del apartado anterior: [reloadKey]. Así, pulsar Volver a intentar hace que React vuelva a ejecutar la carga. Comprueba el estado de carga antes de projects.length para no mostrar un instante de lista vacía.

Práctica Apaga el servidor, observa el error y luego inicia otra vez y pulsa el botón de reintento.

APARTADO 4.7

Crea con POST

Envía los campos en un cuerpo JSON. El servidor devuelve el registro creado y su id; usa esa respuesta para actualizar el estado.

```
export async function createProject(project) {
  const response = await fetch(`${API_URL}/projects`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(project),
  });
  if (!response.ok) throw new Error("No se pudo guardar la idea.");
  return response.json();
}
```

```
async function handleSave(values) {
  setIsSaving(true); setError("");
  try {
    const created = await createProject(values);
    setProjects(items => [...items, created]);
    navigate(`/proyectos/${created.id}`);
  } catch (err) { setError(err.message); }
  finally { setIsSaving(false); }
}
```

Desactiva Guardar mientras espera para evitar envíos dobles. La práctica no anuncia el éxito hasta que JSON Server confirma el guardado.

Práctica Crea una idea de prueba, recarga la página y comprueba que sigue en db.json.

APARTADO 4.8

Edita con PATCH

PATCH pide cambiar solo los campos enviados. Espera la respuesta y actualiza la lista con el proyecto confirmado.

```
export async function updateProject(id, changes) {
  const response = await fetch(`${API_URL}/projects/${id}`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(changes),
  });
  if (!response.ok) throw new Error("No se pudo actualizar.");
  return response.json();
}
```

```
const saved = await updateProject(project.id, formValues);
setProjects(items => items.map(item =>
  item.id === saved.id ? saved : item
));
```

Si falla PATCH, conserva los datos anteriores y muestra el error. PUT suele representar reemplazar un recurso completo; PATCH representa aplicar cambios parciales. Sigue el contrato de la API que estés consumiendo.

Práctica Edita Collar cow print a En curso. Recarga y comprueba que el servidor conserva el cambio.

APARTADO 4.9

Borra con DELETE

Borrar puede ser irreversible. Pide confirmación con el nombre del proyecto y no retires la tarjeta hasta recibir la respuesta.

```
export async function deleteProject(id) {
  const response = await fetch(`${API_URL}/projects/${id}`, { method: "DELETE" });
  if (!response.ok) throw new Error("No se pudo borrar el proyecto.");
}
```

```
async function handleDelete(project) {
  if (!window.confirm(`¿Borrar ${project.title}?`)) return;
  try {
    await deleteProject(project.id);
    setProjects(items => items.filter(item => item.id !== project.id));
  } catch (err) { setError(err.message); }
}
```

No llames response.json() si la API responde sin cuerpo. Si falla el borrado, deja la tarjeta y permite reintentar.

Práctica Borra un proyecto de prueba y confirma que no reaparece al recargar.

APARTADO 4.10

Vuelve a cargar sin copiar el efecto

el apartado 4.6 ya añadió reloadKey y reload. Conserva esas líneas; aquí ajustarás el efecto para que la carga se repita al cambiar reloadKey, sin declarar el estado por segunda vez.

```
const [reloadKey, setReloadKey] = useState(0);
const reload = () => setReloadKey(key => key + 1);
```

El cuerpo del efecto de carga sigue igual que en el apartado 4.5; cambia su lista de dependencias a [reloadKey]. Para una acción como Guardar, llama a la función de servicio desde el evento submit, no desde un efecto que observe una bandera.

Práctica Apaga la API, pulsa reintentar, vuelve a iniciarla y comprueba que el listado se recupera.

APARTADO 4.11

Configura la dirección de la API

En desarrollo y publicación suele cambiar el host. Vite puede leer variables de entorno que comienzan por VITE_.

```
VITE_API_URL=http://localhost:3000
```

Guarda la línea en .env.local en la raíz del proyecto. Luego centraliza la dirección:

```
const API_URL = import.meta.env.VITE_API_URL;
if (!API_URL) throw new Error("Falta VITE_API_URL.");
```

Reinicia Vite tras editar archivos .env. Toda variable VITE_ termina disponible en el navegador: no pongas contraseñas ni claves privadas allí.

Una clave en el cliente no es secreta El código del navegador se descarga en el dispositivo de quien visita la app. Los secretos pertenecen al servidor, no al bundle de React.

Práctica Configura por error el puerto 3001, mira el mensaje de red y vuelve a 3000.

APARTADO 4.12

Filtra resultados con parámetros

Una API puede recibir query parameters y devolver solo una parte de los datos. Conserva el filtro en la URL como aprendiste en la sección 2 y pásalo al servicio.

```
export async function getProjects({ status, signal } = {}) {
  const url = new URL(`${API_URL}/projects`);
  if (status && status !== "Todos") url.searchParams.set("status", status);
  const response = await fetch(url, { signal });
  if (!response.ok) throw new Error("No se pudo filtrar la lista.");
  return response.json();
}
```

JSON Server permite filtrar colecciones mediante query parameters. Otro backend puede usar nombres o reglas diferentes; consulta siempre la documentación del servicio.

Práctica Cambia el filtro y observa la URL de la petición en la pestaña Network. Prueba un estado sin coincidencias.

APARTADO 4.13

Evita que responda una petición antigua

Si se abre una ficha y rápidamente otra, la primera respuesta podría llegar al final. Cancela la lectura anterior y declara el id de ruta como dependencia.

```
useEffect(() => {
  const controller = new AbortController();
  setProject(null); setError("");
  getProject(projectId, { signal: controller.signal })
    .then(setProject)
    .catch(err => {
      if (err.name !== "AbortError") setError(err.message);
    });
  return () => controller.abort();
}, [projectId]);
```

Al cambiar projectId, React limpia el efecto anterior y carga el proyecto nuevo. Una petición de escritura que ya llegó al servidor no se deshace con abort; maneja su respuesta como una acción independiente.

Práctica Cambia rápidamente entre dos fichas y revisa que el título final corresponde al id de la URL.

APARTADO 4.14

Muestra el progreso al guardar

Una petición puede tardar o fallar. Desactiva el botón, presenta un estado Guardando y conserva los datos si el servidor responde con error.

```
<Button type="submit" disabled={isSaving} variant="contained">
  {isSaving ? "Guardando…" : "Guardar cambios"}
</Button>
{saveError && <Alert severity="error">{saveError}</Alert>}
{saved && <Alert severity="success">Proyecto actualizado.</Alert>}
```

Actualiza el estado visible después de la confirmación de la API. Una actualización optimista requiere revertir los datos en caso de fallo, así que empieza con el flujo confirmado por el servidor.

Práctica Usa Network throttling para simular una respuesta lenta. Comprueba que no puedes enviar dos veces.

APARTADO 4.15

Diagnostica peticiones con Network

La pestaña Network muestra URL, método, código de estado, cabeceras, cuerpo y respuesta. Mira esos datos para distinguir errores de interfaz de errores del servidor.

ERR_CONNECTION_REFUSED	El servidor no arrancó o el puerto no coincide.
CORS policy	El servidor no permite el origen de Vite.
404 Not Found	La ruta o el id no existe.
400 o 500	Revisa los datos enviados y los logs del servidor.
La lista no refleja POST	Usa el objeto devuelto por API, incluido el id asignado.

No desactives la protección CORS del navegador. Configura el servidor o un proxy de desarrollo. Un error de CORS y un 404 indican problemas distintos.

Práctica Prueba un puerto sin servidor y luego un id inexistente. Compara el mensaje de red con el estado HTTP.

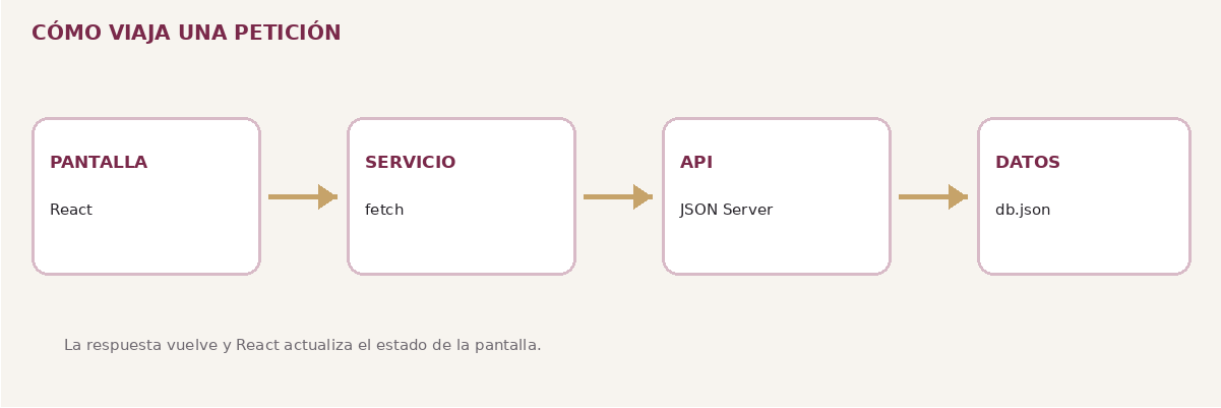
APARTADO 4.16

Proyecto final: conecta el panel

Integra el listado, detalle, creación, edición y borrado con el mismo servicio. Mantén los componentes visuales y las rutas de los secciones anteriores.

Orden de construcción

- Crea db.json y confirma GET /projects.
- Implementa getProjects y estados de carga/error.
- Conecta detalle con el id de React Router.
- Añade POST a Nueva idea.
- Añade PATCH a Editar idea.
- Añade DELETE con confirmación.
- Prueba búsqueda, filtros, recarga y datos persistentes.



La pantalla delega las peticiones al servicio; la API administra los datos.

Comprobación final Crea una idea, recarga, edítala y bórrala. Anota qué endpoint ejecuta cada paso.

APARTADO 4.17

Resuelve problemas frecuentes

Cuando algo no funciona, revisa primero la pestaña Network y después la consola del servidor.

La página se queda cargando	Revisa finally y si el servidor responde.
El borrado causa error JSON	DELETE quizá no devuelve cuerpo; no lo conviertas.
El elemento nuevo no tiene id	Usa el id que devuelve el servidor, no uno inventado en React.
La edición parece exitosa pero se pierde	Actualiza con el objeto confirmado y revisa PATCH.
El filtro no modifica la petición	Comprueba qué valor llega desde la URL y al servicio.
CORS entre dos puertos	Configura servidor/proxy; no apagues la seguridad del navegador.

Lista de comprobación

- ☐ Abro la API en :3000 y la app Vite en su puerto.
- ☐ Veo estados de carga, error y lista vacía.
- ☐ Puedo crear, editar y borrar un registro ficticio.
- ☐ Recargo y confirmo que la API conserva los cambios.
- ☐ Reviso método, ruta y respuesta en Network.

Bitácora Anota qué detalle viste en Network para localizar un error de API.

APARTADO 4.18

Repaso y siguientes pasos

La interfaz presenta estados, el servicio centraliza peticiones y la API valida y guarda los datos. Esta estructura hará más sencillo añadir caché, permisos y despliegue en los siguientes pasos.

- ☐ Puedo explicar cliente, servidor, endpoint, HTTP y JSON.
- ☐ Sé usar GET, POST, PATCH y DELETE.
- ☐ Compruebo response.ok antes de leer una respuesta.
- ☐ Mantengo fetch y URLs fuera de mis páginas.
- ☐ Muestro carga, error, vacío, éxito y reintento.
- ☐ Cancelo lecturas que ya no necesita la pantalla.
- ☐ No pongo claves privadas en variables VITE_.
- ☐ Uso Network para comprobar peticiones y respuestas.

Glosario

API — Contrato para pedir o modificar datos de otro sistema.

Endpoint — Dirección de un recurso, por ejemplo /projects/1.

JSON — Formato de texto para intercambiar objetos y listas.

CRUD — Crear, leer, actualizar y borrar recursos.

CORS — Regla del navegador para solicitudes entre distintos orígenes.

AbortController — API del navegador para cancelar una petición fetch pendiente.

Siguiente sección. La parte final reúne pruebas, accesibilidad, configuración, compilación y publicación del panel.

SECCIÓN 5

Calidad y publicación

Prueba los comportamientos importantes, revisa accesibilidad y prepara un build de producción.

Al terminar esta sección

Al terminar podrás ejecutar pruebas, comprobar rutas de una SPA y entender qué falta para publicar una API real.

Apartados

5.1 Qué significa cuidar la calidad
5.2 Elige qué tipo de prueba necesitas
5.3 Instala las herramientas de prueba
5.4 Configura Vitest junto a Vite
5.5 Escribe tu primera prueba de formulario
5.6 Comprueba validación y mensajes
5.7 Usa consultas que reflejen la interfaz
5.8 Prueba una respuesta asíncrona
5.9 Revisa la accesibilidad con teclado
5.10 Asocia etiquetas e instrucciones
5.11 Revisa errores, foco y contraste
5.12 Ejecuta el build de producción
5.13 Prepara las rutas para una SPA
5.14 Separa variables locales y de publicación
5.15 Publica una versión de prueba
5.16 Diagnostica la versión publicada
5.17 Prepara una lista de salida
5.18 Cierre de la serie y glosario

APARTADO 5.1

Qué significa cuidar la calidad

La calidad se nota cuando la interfaz responde a lo que la persona espera, conserva los datos correctos y ofrece una salida clara cuando algo falla. No depende solo de que la app compile.

Prueba automática	Comprueba de manera repetible una regla o interacción.
Revisión manual	Permite valorar claridad, teclado, contraste y flujo completo.
Build	Comprueba que el proyecto puede transformarse en archivos de producción.
Preview	Permite probar localmente esa versión compilada.

Prueba primero los comportamientos importantes: crear una idea válida, explicar un error de validación y mostrar la ruta adecuada. Esos ejemplos aportan más confianza que probar cada detalle de implementación.

Práctica Escribe tres cosas que sería frustrante que dejaran de funcionar en tu app madebm3.

APARTADO 5.2

Elige qué tipo de prueba necesitas

Una prueba de componente renderiza parte de la interfaz y simula acciones. Una prueba de extremo a extremo recorre la app en un navegador real. Esta guía empezará con componentes; la revisión manual completará los flujos grandes.

Regla o función	Prueba un resultado para una entrada concreta.
Componente	Prueba lo que aparece y cómo responde a una acción.
Aplicación completa	Prueba rutas y servicios como lo haría una persona.

Evita comprobar detalles internos como el nombre de un estado de React. A quien usa la app le importa que Guardar añada la idea, no si la función se llama handleSubmit.

No necesitas probarlo todo hoy Empieza por reglas que pueden romper la experiencia o los datos. Amplía la cobertura a medida que aparezcan errores reales o funciones nuevas.

Práctica Para cada uno de los tres tipos de prueba, elige un ejemplo concreto del panel.

APARTADO 5.3

Instala las herramientas de prueba

Vitest ejecuta las pruebas con una configuración compatible con proyectos Vite. Testing Library ayuda a interactuar con los componentes mediante sus nombres y roles accesibles.

```
npm install --save-dev vitest jsdom @testing-library/react @testing-library/dom @testing-library/user-event
@testing-library/jest-dom
```

jsdom proporciona un entorno de navegador simulado para las pruebas. React Testing Library renderiza componentes; user-event simula acciones como escribir o pulsar. Añadiremos un matcher que hace las aserciones del DOM más fáciles de leer.

Añade un script sin borrar los demás

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "test": "vitest"
}
```

Conserva api y preview si ya aparecen en package.json. Usa npm run test para abrir el modo de observación mientras programas, y npm run test -- --run para ejecutar una sola pasada.

Práctica Instala las herramientas y comprueba que npm run test abre Vitest sin errores.

APARTADO 5.4

Configura Vitest junto a Vite

Vitest puede usar el archivo de configuración de Vite. Conserva el plugin React que ya estaba y añade las opciones de prueba:

```
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  test: {
    environment: "jsdom",
    setupFiles: "./src/test/setup.js",
    clearMocks: true,
  },
});
```

Si tu vite.config.js ya contiene alias, plugins u otras opciones, combínalas en el mismo objeto; no crees dos configuraciones que compitan. Crea src/test/setup.js:

```
import "@testing-library/jest-dom/vitest";
```

El entorno jsdom ofrece document y elementos DOM en Node. La app final sigue ejecutándose en el navegador normal; este entorno solo existe al correr las pruebas.

Práctica Crea un archivo src/components/ProjectForm.test.jsx. Añade una prueba vacía y comprueba que Vitest la encuentra.

APARTADO 5.5

Escribe tu primera prueba de formulario

Una buena prueba reproduce una interacción. Usa el rol del botón y el nombre visible del campo, escribe algo y comprueba el resultado que puede percibir la persona.

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { expect, test, vi } from "vitest";
import ProjectForm from "./ProjectForm";

test("guarda el título escrito", async () => {
  const user = userEvent.setup();
  const onSave = vi.fn();
  render(<ProjectForm onSave={onSave} />);

  await user.type(screen.getByLabelText("Nombre del proyecto"), "Agenda de otoño");
  await user.click(screen.getByRole("button", { name: "Guardar" }));

  expect(onSave).toHaveBeenCalledWith(
    expect.objectContaining({ title: "Agenda de otoño" })
  );
});
```

Adapta la etiqueta y el nombre del botón a tu formulario real. La prueba usa la interfaz como la encuentra una persona, no busca el input por una clase CSS interna.

Práctica Ejecuta la prueba y lee el mensaje de Vitest. Cambia temporalmente el nombre del botón esperado para comprobar que una prueba fallida señala el problema.

APARTADO 5.6

Comprueba validación y mensajes

Prueba el caso vacío: la persona envía el formulario y recibe una explicación, mientras la función onSave no se ejecuta.

```
test("explica que falta el título", async () => {
  const user = userEvent.setup();
  const onSave = vi.fn();
  render(<ProjectForm onSave={onSave} />);

  await user.click(screen.getByRole("button", { name: "Guardar" }));

  expect(screen.getByRole("alert")).toHaveTextContent("Escribe un título");
  expect(onSave).not.toHaveBeenCalled();
});
```

Para que getByRole("alert") encuentre el error, la app debe marcar el mensaje como role="alert" o usar un componente accesible que anuncie el error. Cada prueba útil ayuda a comprobar si la interfaz ofrece la semántica que espera.

Una prueba por comportamiento

Mantén la prueba de éxito y la de error separadas. Si una falla, sabrás qué conducta investigar. No uses tiempos arbitrarios como setTimeout para esperar; las consultas asíncronas de Testing Library están pensadas para contenido que aparece después.

Práctica Añade una prueba para un título de dos caracteres y otra para un título válido. Comprueba que el error se elimina al corregirlo si ese es el comportamiento de tu formulario.

APARTADO 5.7

Usa consultas que reflejen la interfaz

Testing Library propone buscar de formas parecidas a cómo se localiza un control: por rol, texto, etiqueta o nombre accesible. Prefiere estas consultas a selectores de implementación.

getByRole	Busca un botón, encabezado, enlace o región por su rol.
getByLabelText	Busca un campo por la etiqueta asociada.
getByText	Busca texto visible cuando no hay una consulta semántica mejor.
findByRole	Espera a que aparezca un elemento asíncrono.
queryByRole	Comprueba que algo no está presente sin lanzar error.

```
expect(screen.getByRole("heading", { name: "Mis proyectos" })).toBeInTheDocument();
expect(screen.queryByRole("alert")).not.toBeInTheDocument();
```

Una etiqueta ausente suele ser un problema para usuarios y hace difícil probar el control. Si una consulta por label falla, revisa la interfaz antes de recurrir a un selector frágil.

Práctica Sustituye una búsqueda por className en una prueba por getByRole o getByLabelText.

APARTADO 5.8

Prueba una respuesta asíncrona

El contenido remoto no aparece en el mismo instante. Usa findBy... cuando esperas que un elemento aparezca y configura una respuesta controlada para no depender de internet.

```
expect(await screen.findByRole("heading", { name: "Agenda académica" }))
  .toBeInTheDocument();
```

En esta primera serie de pruebas puedes validar el formulario y tarjetas sin red. Las pruebas de fetch requieren simular el servicio o interceptar peticiones; evita llamar a la API real en cada ejecución para mantener resultados rápidos y repetibles.

Prueba estable Cada test debe preparar los datos que necesita y no depender del estado de db.json o de una conexión externa.

Práctica Simula en una prueba el estado sin resultados y verifica que aparece un mensaje claro.

APARTADO 5.9

Revisa la accesibilidad con teclado

Usa Tab para recorrer enlaces y botones, Shift+Tab para volver y Enter o Espacio para activar. El indicador de foco debe permanecer visible y el orden debe seguir la lectura natural de la pantalla.

- ☐ Los enlaces navegan y los botones ejecutan acciones.
- ☐ Los campos tienen una etiqueta visible y asociada.
- ☐ Los errores dicen qué corregir y están cerca del control.
- ☐ El foco no desaparece al abrir un menú o un diálogo.
- ☐ El color no es la única forma de identificar estado o error.
- ☐ El zoom del navegador no oculta botones ni contenido importante.

Realiza esta revisión en listado, detalle y formulario. Las pruebas automáticas pueden detectar algunas omisiones, pero no sustituyen recorrer la app con teclado y valorar si las instrucciones se entienden.

Práctica Cierra el ratón y completa alta, edición y regreso al listado solo con teclado.

APARTADO 5.10

Asocia etiquetas e instrucciones

Cada control necesita un nombre que explique su propósito. Las etiquetas visibles ayudan a entender y activar el campo, y permiten que la tecnología de asistencia lo identifique.

```
<label htmlFor="project-title">Nombre del proyecto</label>
<input id="project-title" name="title"
  aria-describedby="title-help" />
<p id="title-help">Usa al menos 3 caracteres.</p>
```

Con MUI, TextField label="Nombre del proyecto" mantiene una etiqueta visible asociada. Para controles propios, asocia label con id o usa la API accesible del componente. Evita usar placeholder como sustituto de una etiqueta persistente.

Orden de encabezados

Conserva un h1 que describa la pantalla y usa h2 para secciones. El diseño visual puede ajustar el tamaño, pero la etiqueta HTML debe representar la estructura del contenido.

Práctica Inspecciona cada campo con getByLabelText y comprueba que la etiqueta coincide con el texto visible.

APARTADO 5.11

Revisa errores, foco y contraste

Cuando el envío falla, muestra texto que explique la corrección. Si un error solo cambia el borde a rojo, algunas personas no podrán percibir qué ocurrió. Coloca el foco donde la persona pueda continuar la tarea.

Comprobaciones manuales

- ☐ Envía el formulario vacío y localiza el primer campo que necesita corrección.
- ☐ Lee el mensaje de error sin depender solo del color.
- ☐ Comprueba contraste de texto y foco en fondos claros y oscuros.
- ☐ Haz zoom y comprueba que los campos no quedan ocultos.
- ☐ Prueba títulos largos en tarjetas y navegación móvil.

Un verificador automático puede señalar problemas, pero no entiende si un mensaje es útil ni si el orden de lectura tiene sentido. Usa la revisión como guía para encontrar puntos que luego confirmas manualmente.

Práctica Cambia temporalmente el color de error a un tono parecido al fondo. ¿Sigue siendo claro el texto y visible el estado?

APARTADO 5.12

Ejecuta el build de producción

El servidor de desarrollo ayuda a programar, pero no es la entrega final. npm run build pide a Vite que genere los archivos estáticos de producción dentro de dist.

```
npm run build
```

El comando suele estar configurado como vite build en package.json. Lee el primer error de compilación y corrige referencias rotas, importaciones incorrectas o variables ausentes antes de continuar.

Prueba la carpeta compilada

```
npm run preview
```

El script preview suele apuntar a vite preview. Abre la dirección que muestra la terminal y prueba la app como una persona. Este servidor es para revisar localmente; no es el servidor que publica la web en internet.

Práctica Genera dist, inicia preview y abre Inicio, Proyectos y Detalle. Comprueba que la compilación sirve archivos y rutas esperadas.

APARTADO 5.13

Prepara las rutas para una SPA

React Router cambia la pantalla en el navegador. Si alguien abre directamente /proyectos/1, el alojamiento debe servir index.html para que la app lea esa dirección.

Algunos alojamientos configuran ese fallback automáticamente. En otros puedes añadir una regla de reescritura. Un formato usado por ciertos proveedores es crear public/_redirects:

```
/*    /index.html   200
```

Vite copia los archivos de public a dist. Cada plataforma usa su propio mecanismo; verifica la guía de tu proveedor. Un 404 solo al recargar una ruta suele indicar que falta el fallback de la SPA.

Práctica Prueba abrir directamente la URL del detalle desde preview y recarga. Si falla, identifica qué regla debería aplicar el alojamiento.

APARTADO 5.14

Separa variables locales y de publicación

La API de la sección 4 en localhost solo existe en tu ordenador. Al publicar el frontend, esa dirección no será accesible para otras personas.

Antes de poner el panel en internet

- ☐ Sustituye JSON Server por una API alojada y mantenida.
- ☐ Configura la variable VITE_API_URL en el alojamiento.
- ☐ Permite solicitudes desde el dominio publicado en el servidor API.
- ☐ No incluyas claves privadas en variables VITE_.
- ☐ Comprueba permisos, validación y datos en el backend.

Las variables VITE_ se incorporan al bundle y son visibles para quien descarga la app. Una autenticación real y la protección de datos requieren decisiones del servidor que esta guía de frontend no sustituye.

No publiques datos reales de práctica Usa proyectos ficticios hasta configurar almacenamiento, permisos y privacidad en un backend adecuado.

Práctica Escribe cuál sería la dirección de API local y qué dato tendría que cambiar al publicar.

APARTADO 5.15

Publica una versión de prueba

El proveedor de alojamiento suele pedir el comando de build y el directorio que contiene los archivos listos. En Vite, los valores comunes son npm run build y dist.

Comando de construcción	npm run build
Directorio de salida	dist
Rutas de React Router	Configura fallback al index.html si hace falta.
Variables de entorno	Define VITE_API_URL en el panel del proveedor.
API	Debe estar desplegada, responder y permitir el dominio de la app.

La interfaz puede alojarse como archivos estáticos, pero sus datos dinámicos requieren una API disponible. Desplegar solo dist no publica JSON Server ni crea una base de datos en internet.

Práctica Prepara una versión de prueba en un entorno privado o temporal. Abre cada ruta directamente y revisa los errores del navegador.

APARTADO 5.16

Diagnostica la versión publicada

Si funciona en localhost pero falla en publicación, compara rutas, variables, origen de la petición y contenido de dist. La consola y Network muestran si el problema ocurre al descargar archivos o al pedir datos.

Pantalla en blanco	Busca error JavaScript en Console.
404 al recargar una ruta	Configura el fallback SPA al index.html.
API no responde	Revisa la dirección VITE_API_URL y el estado del backend.
CORS al pedir datos	Permite el dominio correcto en el servidor API.
Assets no aparecen en subruta	Comprueba la opción base de Vite y la ruta de publicación.
Funciona en preview, no en host	Compara ajustes de rutas y reglas del proveedor.

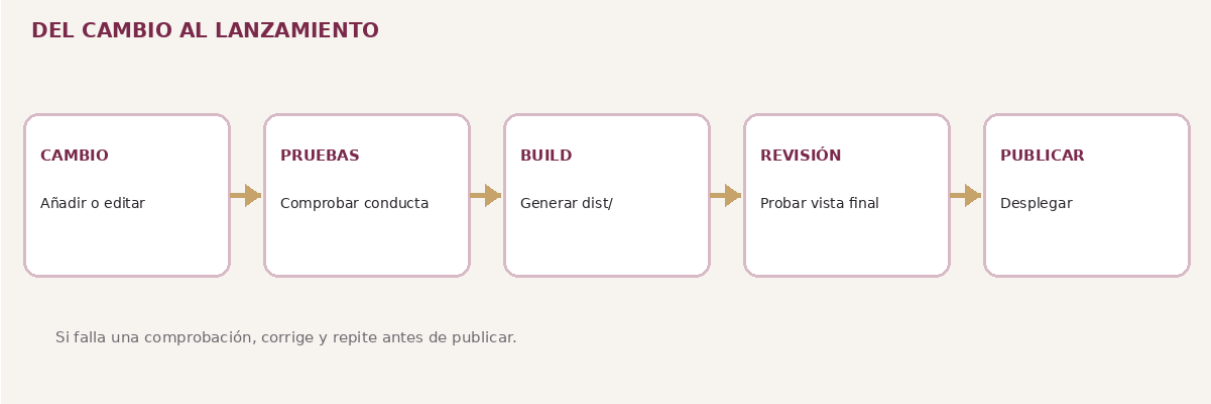
Práctica Anota un fallo de publicación y qué pestaña del navegador usarías primero para investigarlo.

APARTADO 5.17

Prepara una lista de salida

Antes de considerar la app lista, combina pruebas automáticas y una comprobación del flujo completo. No hace falta marcar todo si el proyecto no usa esa función; registra lo que comprobaste.

- ☐ npm run test -- --run termina sin pruebas fallidas.
- ☐ npm run build genera dist sin errores.
- ☐ npm run preview abre las rutas principales.
- ☐ Formulario válido guarda; inválido explica cómo corregirlo.
- ☐ La app funciona con teclado y mantiene foco visible.
- ☐ La API de publicación no usa localhost.
- ☐ El dominio de la app está autorizado por el backend.
- ☐ No hay claves privadas en el código que recibe el navegador.
- ☐ Borrado y edición se prueban con datos de prueba.



Repite las comprobaciones relevantes después de corregir un fallo.

Registro de lanzamiento Fecha: _____ Versión probada: _____ Pendientes: _____

APARTADO 5.18

Cierre de la serie y glosario

Has recorrido el camino desde JSX hasta una app que puede probarse y prepararse para publicación. El proyecto madebm3 puede seguir creciendo con un backend seguro, mejores pruebas y un diseño propio.

- ☐ Sé escribir una prueba centrada en lo que hace la persona.
- ☐ Uso roles y etiquetas para localizar controles.
- ☐ Puedo revisar navegación y formularios con teclado.
- ☐ Distingo el build de producción del servidor de desarrollo.
- ☐ Entiendo por qué las rutas SPA necesitan fallback.
- ☐ Sé que la API local de prueba no es una API pública.
- ☐ No publico secretos en el bundle de React.

Glosario

Prueba de componente — Comprueba la interfaz de una pieza React y una interacción.

jsdom — Entorno DOM simulado para pruebas ejecutadas en Node.

getByRole — Consulta un elemento por su rol semántico y nombre accesible.

Build — Archivos optimizados que Vite genera para producción.

Preview — Servidor local para comprobar el build.

Fallback SPA — Regla que sirve index.html al entrar directamente en una ruta cliente.

Siguiente mejora Una vez que la app funciona, elige una mejora concreta para madebm3: integrar una API de verdad, publicar en un dominio propio o añadir una vista móvil más específica.

Referencias oficiales

Estas páginas sustentan los ejemplos técnicos reunidos en el manual. Consulta la documentación de la versión instalada cuando reutilices comandos o APIs.

React Learn — https://react.dev/learn

React Installation — https://react.dev/learn/installation

React: Build a React app from scratch — https://react.dev/learn/build-a-react-app-from-scratch

Vite Getting Started — https://vite.dev/guide/

React Router · Instalación declarativa — https://reactrouter.com/start/declarative/installation

React Router · Rutas — https://reactrouter.com/start/declarative/routing

React Router · Navegación — https://reactrouter.com/start/declarative/navigating

React Router · Valores de URL — https://reactrouter.com/start/declarative/url-values

React · Compartir estado entre componentes — https://react.dev/learn/sharing-state-between-components

React · Reaccionar a las entradas con estado — https://react.dev/learn/reacting-to-input-with-state

Material UI · Instalación — https://mui.com/material-ui/getting-started/installation/

Material UI · Uso — https://mui.com/material-ui/getting-started/usage/

Material UI · Tema — https://mui.com/material-ui/customization/theming/

Material UI · Button — https://mui.com/material-ui/react-button/

Material UI · Stack — https://mui.com/material-ui/react-stack/

Material UI · Grid — https://mui.com/material-ui/react-grid/

Material UI · Diseño adaptable — https://mui.com/material-ui/guides/responsive-ui/

React · Sincronizar con efectos — https://react.dev/learn/synchronizing-with-effects

React · Referencia useEffect — https://react.dev/reference/react/useEffect

React · No siempre necesitas un efecto — https://react.dev/learn/you-might-not-need-an-effect

JSON Server 0.17.4 · README — https://github.com/typicode/json-server/blob/v0.17.4/README.md

MDN · fetch — https://developer.mozilla.org/docs/Web/API/Fetch_API/Using_Fetch

MDN · AbortController — https://developer.mozilla.org/docs/Web/API/AbortController

Vitest · Getting Started — https://vitest.dev/guide/

Vitest · Entorno de prueba — https://vitest.dev/guide/environment

Testing Library · React Testing Library — https://testing-library.com/docs/react-testing-library/intro/

Testing Library · user-event — https://testing-library.com/docs/user-event/intro/

jest-dom · Uso con Vitest — https://github.com/testing-library/jest-dom#with-vitest

W3C WAI · Etiquetas de controles — https://www.w3.org/WAI/tutorials/forms/labels/

W3C WAI · Revisión inicial de accesibilidad — https://www.w3.org/WAI/test-evaluate/preliminary/

Vite · Build para producción — https://vite.dev/guide/build

Vite · Despliegue estático — https://vite.dev/guide/static-deploy

React Router · SPA y fallback — https://reactrouter.com/how-to/spa